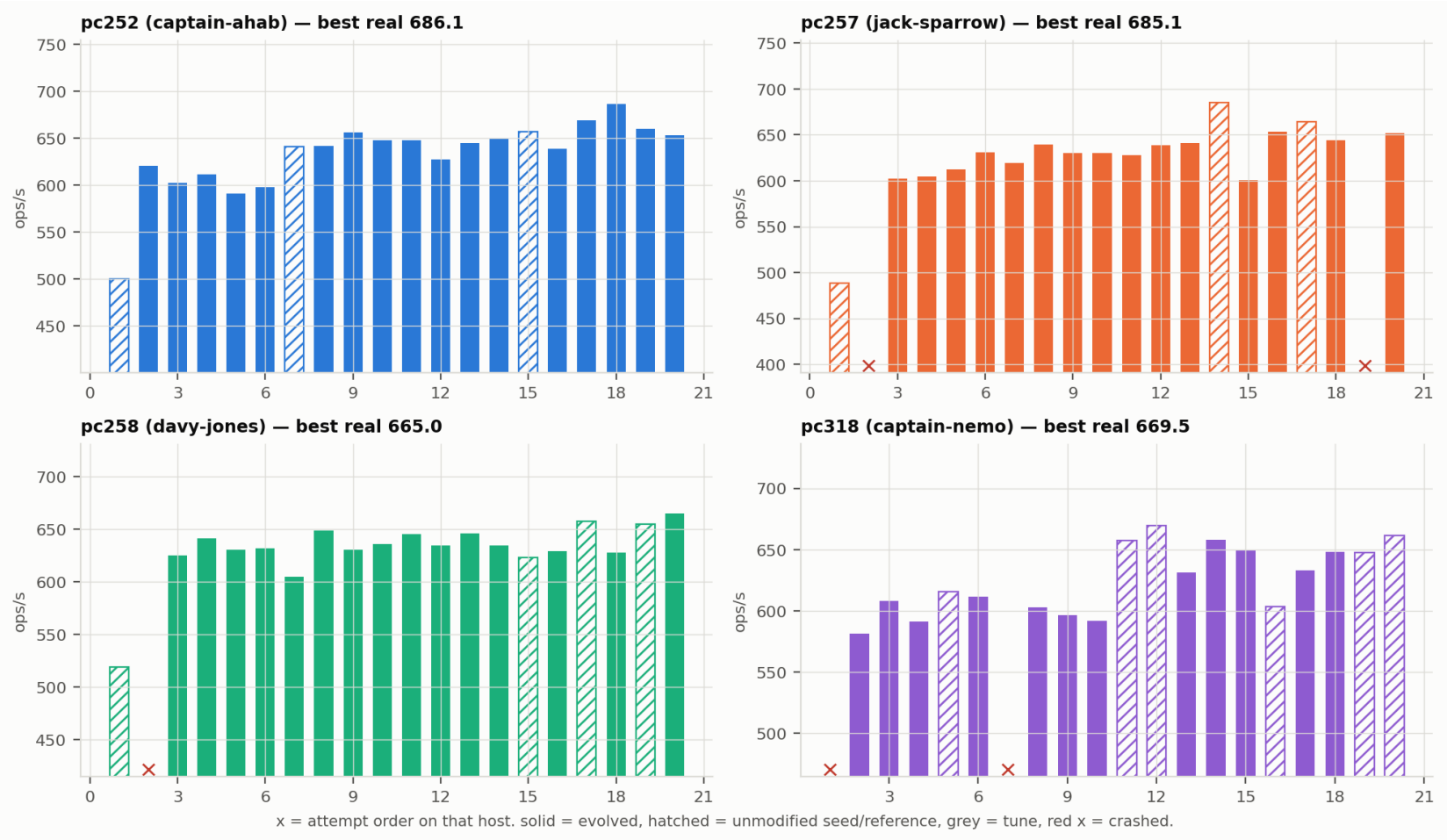
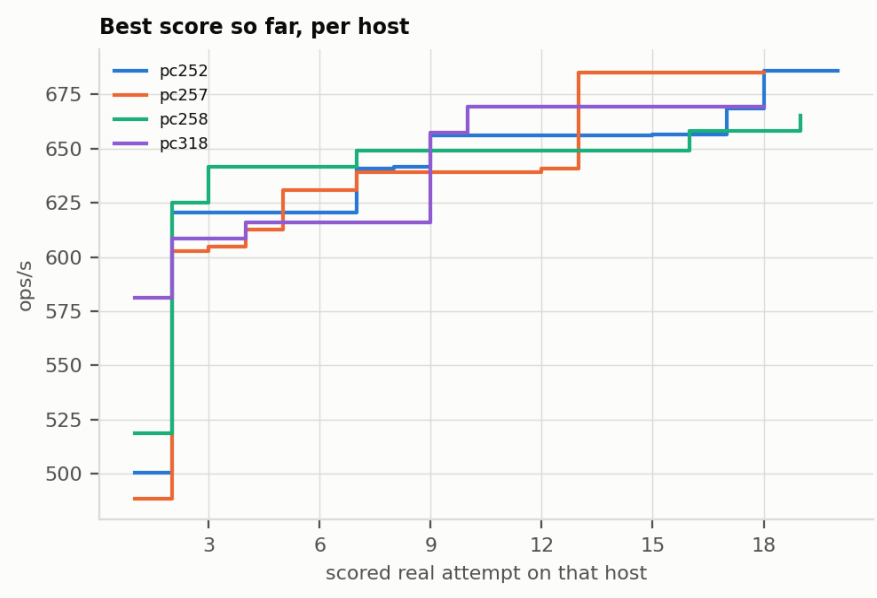
Multi-host CORAL evolution — run 2026-10-26_195014

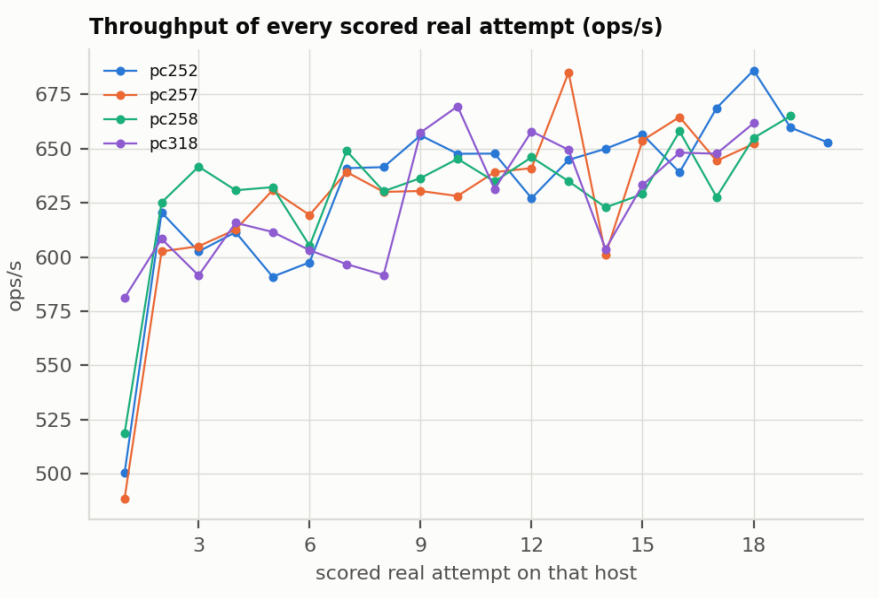
4 agents, one per CloudLab host (each agent evaluated only on its own host, in parallel); score = absolute measured throughput (ops/s). Budget 20 real evaluations per host; all attempts are in one shared leaderboard.

host	agent	real / tune / crashed	best score	its tput (ops/s)	max tput (ops/s)	host no-op baseline (ops/s, n=8; tanh only)	LLM cost
pc252	captain-ahab	20 / 0 / 0	#70 69132a21 686.1	686.1	686.1	-	$5
pc257	jack-sparrow	20 / 0 / 2	#56 317ee824 685.1	685.1	685.1	-	$5
pc258	davy-jones	20 / 0 / 1	#72 2e7dd61d 665.0	665.0	665.0	-	$5
pc318	captain-nemo	20 / 0 / 2	#53 35f7f9f5 669.5	669.5	669.5	-	$6

- Overall best: pc252 (captain-ahab), #70 69132a21, score 686.1, 686.1 ops/s — agent-evolved.
- Run stop: max_real_attempts; wall time 0.81 h; total LLM cost $21; gateway requests {400: 122, 200: 686}.
- Hosts differ in speed, so raw throughput is not comparable across hosts: compare changes within a host, not levels across hosts.







Top 10 attempts across all hosts (real, scored)

#	host	agent	attempt	commit	score	tput	MiB read	refaults	design / title
1	pc252	captain-ahab	70	69132a21	686.1	686.1	193	23845	MGLRU tier sweep, end point: tier protection DISABLED entirely (TIER_PROTECT 0: oldest-gen folios evicted regardless of refs; only
2	pc257	jack-sparrow	56	317ee824	685.1	685.1	195	24342	CROSS-HOST REPLICATION: reference/mglru.c unchanged (+1 comment) on pc257, per captain-nemo (657.4/669.5 on pc318 vs noop 615.7).
3	pc318	captain-nemo	53	35f7f9f5	669.5	669.5	192	23479	REPLICATION: reference mglru.c unchanged + 1 comment line (prev 657.4 on pc318 vs noop 615.7). Confirms or kills the +6.8% before
4	pc252	captain-ahab	67	22bb52af	668.6	668.6	191	23451	structural 3/3 MGLRU lane: tier_threshold fixed at MAX_NR_TIERS-1=3 (only folios with >=16 refs tier-protected; ~pure generational
5	pc258	davy-jones	72	2e7dd61d	665.0	665.0	189	22974	FINAL eval, CROSS-HOST REPLICATION of captain-ahab 22bb52af (MGLRU, tier_threshold fixed at 3 = near-pure generational aging; 668.
6	pc257	jack-sparrow	66	ae2a0c1b	664.5	664.5	192	23701	REPLICATION #2 of reference mglru.c on pc257 (first run 685.06; comment-only change). Needed because both my mglru perturbations l
7	pc318	captain-nemo	80	a8531cd7	661.7	661.7	191	23369	FINAL eval: noop replicate #2 on pc318 (seed noop.c + comment) to tighten the baseline under the MGLRU claim (noop #1 615.74; MGLR
8	pc252	captain-ahab	74	d4aba8dc	659.6	659.6	193	23915	REPLICATE-ish of 69132a21 (no tier protection = 686.07, best; +7% vs same-host noop 641): identical eviction behavior, but folio_a
9	pc258	davy-jones	59	479412c6	657.9	657.9	188	22912	CROSS-HOST: reference/mglru.c unchanged on pc258 (nemo: 657/669 on pc318 vs noop 616). pc258 noop=622.9, my best 372e2b1f=648.9/64
10	pc318	captain-nemo	61	45a24f02	657.9	657.9	202	25813	structural 2/3 on MGLRU: reference mglru.c + never-read reserve (newest 512 insertions with refs<=1 moved to next gen instead of e

All attempts on pc252 (captain-ahab)

#	commit	class	score	tput	read MiB	refaults	walls	design / title
1	b5d6b935	real =get_scan	500.4	500.4	136	11098	41	baseline: reference/get_scan.c unchanged (scan-pid list + sampled LFU) for calibration on my host
2	ee0fb848	real	620.5	620.5	170	18437	41	structural 1/3 scan-offload: ONLY scan_pids-inserted folios on our FIFO list (exact count via small hash), evict oldest scan folios first ke
3	b03fc887	real	602.5	602.5	170	18655	41	scan-offload 2/3 (ablation of ee0fb848): SCAN_RESERVE 256->2048 pages. Hypothesis: ee0fb848 refaults jumped to 18.4k (vs 11.1k get_scan) bec
4	572e5ce9	real	611.4	611.4	167	17841	41	scan-offload structural 3/3 (new sub-idea: readahead-junk filter). Reverted reserve to 256 (2048 gave 602 w/ unchanged refaults 18.7k -> res
5	35adbc66	real	590.9	590.9	159	16138	41	REVIEW/replication: davy-jones 378d80f2 (scan-first freq-CLOCK, team top 641.66 on pc258) resubmitted UNCHANGED (one comment line added) on
6	486f0369	real	597.5	597.5	162	16796	41	calibration: exact seed noop.c on MY host pc252 (+1 comment line). Same-host baseline for my 620/602/611 scan-offload and 591 replication of
7	3ad7008b	real =noop	641.0	641.0	181	20789	41	calibration: exact seed noop.c (from 55919a84) on MY host pc252. CORRECTION: my previous eval 76?/'calibration: exact seed noop' was mislabe
8	2d090442	real	641.4	641.4	180	20435	41	DIAGNOSTIC: attached no-op. struct_ops attached, all hooks empty, evict_folios proposes 0 victims (kernel fallback does everything). KEY CON
9	b87946a9	real	656.0	656.0	178	20043	41	FINDING + test. attached no-op 2d090442 = 641.42 ~= seed noop 640.96 on pc252 => attaching cache_ext costs NOTHING; the 3-8% losses come fro
10	04ab0e18	real	647.6	647.6	177	19848	41	REPLICATE b87946a9 (probation-only junk filter, 656.05) unchanged + comment, to test whether +2.3% over same-host noop pair (640.96/641.42)
11	7280f092	real	647.7	647.7	168	17948	41	never-accessed-first for ALL folios (1-line change from b87946a9): scan-thread folios now also enter probation and are released to kernel on
12	36b60722	real	627.1	627.1	176	19728	41	knob on b87946a9 (GET-only junk filter, 656/648): PROBE_RESERVE 512->32. Hypothesis: gain scales with junk freed; 512 kept ~2MiB (~4% of cac
13	95fb87da	real	644.8	644.8	179	20338	41	knob shape: PROBE_RESERVE 2048 on b87946a9 base. Reflection on 36b60722 (reserve 32 = 627, -25 vs 512's mean 651.8, below noop 641): FALSIFI
14	ea09d75e	real	649.9	649.9	179	20183	41	knob: PROBE_RESERVE 1024 on b87946a9 base (refine peak). Reflection on 95fb87da (2048 = 644.75 ~ noop): curve 32->627, 512->656/648, 2048->6
15	1a10b4c4	real =mglru	656.5	656.5	194	24018	41	structural 1/3 MGLRU lane: reference/mglru.c UNCHANGED (+comment) on pc252, cross-host replication of nemo's 657/669 on pc318 (+7.7% vs his
16	36f75f6a	real	639.0	639.0	169	18325	41	structural 2/3 MGLRU lane: fixed tier_threshold=0 (replaces PID-controller get_tier_idx) => any folio re-accessed after insertion is promote
17	22bb52af	real	668.6	668.6	191	23451	40	structural 3/3 MGLRU lane: tier_threshold fixed at MAX_NR_TIERS-1=3 (only folios with >=16 refs tier-protected; ~pure generational aging). R
18	69132a21	real	686.1	686.1	193	23845	41	MGLRU tier sweep, end point: tier protection DISABLED entirely (TIER_PROTECT 0: oldest-gen folios evicted regardless of refs; only locked/wr
19	d4aba8dc	real	659.6	659.6	193	23915	41	REPLICATE-ish of 69132a21 (no tier protection = 686.07, best; +7% vs same-host noop 641): identical eviction behavior, but folio_accessed ho
20	ee3d81d1	real	652.9	652.9	190	23031	40	FINAL (20/20): exact replicate of 69132a21 (MGLRU, tier protection disabled, folio_accessed hook kept) = 686.07 first run. Purpose: separate

All attempts on pc257 (jack-sparrow)

#	commit	class	score	tput	read MiB	refault s	wall s	design / title
1	5bb6ccaa	real =get_scan	488.6	488.6	133	10619	41	baseline: reference get_scan.c unchanged (calibrate my machine; scan-list sampled LFU)
2	d02fff5d	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
3	bceb7742	real	602.5	602.5	168	17895	41	structural 1/3 freq-history: scan-first FIFO (256 reserve) + S3-FIFO-style small/main for GET folios, but frequency is a persistent per-(ino
4	3e1b84bb	real	604.9	604.9	172	18772	40	structural 2/3 freq-history (parent bceb7742=602.55): scan list now evicts only folios the scan thread already READ (consumed flag set in fo
5	e4db7979	real	612.6	612.6	179	20573	41	structural 3/3 freq-history: ablate scan-first list (parent 3e1b84bb=604.9). Scan-thread folios enter the S3-FIFO small queue like GET folio
6	14bafa84	real	630.8	630.8	170	18458	41	freq-history + junk-first (parent e4db7979=612.6, no scan-first list): before normal small-queue S3-FIFO eviction, a pass evicts GET-inserte
7	fba686fd	real	619.3	619.3	172	19027	40	junk-first 2/3 (parent 14bafa84=630.8): also run the never-accessed-since-insert eviction pass over the main list (ghost/history-admitted re
8	fec3a52f	real	639.2	639.2	173	19081	41	REPLICATION of 14bafa84 (630.8; junk-first small pass only), code identical except a comment. Eval fba686fd (+main-list junk pass) gave 619.
9	5b6cc6a1	real	630.0	630.0	174	19563	40	junk-first 3/3, O(1) version (parent 14bafa84 replicated 630.8/639.2): GET-inserted folios without history go to a separate JUNK list; first
10	936b9cff	real	630.4	630.4	174	19458	41	knob on junk-list (6bd? parent=630.0): EPOCH_SHIFT 14->16, freq history halves every 65K GET accesses (~1x per run) instead of every 16K. Wo
11	f2082dc2	real	628.1	628.1	159	15970	41	CROSS-HOST REPLICATION of davy-jones 372e2b1f (scan-first freq-CLOCK + probation FIFO; 648.9/645.3 on pc258) on pc257, comment-only change.
12	dbc8b691	real	639.1	639.1	180	20779	41	knob on my junk-list+freq-history (936b9cff=630.4, EPOCH_SHIFT 16): small (probation, real once-read pages only) FIFO 10% -> 25% of cgroup p
13	ab39bb80	real	641.0	641.0	176	19811	41	sweep small FIFO size on junk-list+freq-history: 10%=630.4, 25%=639.1 -> now 50% (cache_size/2; main<=small guard also caps). Testing whethe
14	317ee824	real =mglru	685.1	685.1	195	24342	40	CROSS-HOST REPLICATION: reference/mglru.c unchanged (+1 comment) on pc257, per captain-nemo (657.4/669.5 on pc318 vs noop 615.7). My best on
15	b80da597	real	601.0	601.0	187	22536	41	mglru 1/3 (parent = reference mglru.c, 685.06 on pc257): lru_gen_add_folio uses list_add_tail instead of list_add, so new folios sit at the
16	fbc1bb56	real	653.8	653.8	192	23599	41	mglru 2/3 (parent reference mglru.c = 685.06): new non-active folios always go to the OLDEST gen (seq=min_seq) instead of min_seq+1 when >2
17	ae2a0c1b	real =mglru	664.5	664.5	192	23701	41	REPLICATION #2 of reference mglru.c on pc257 (first run 685.06; comment-only change). Needed because both my mglru perturbations lost (tail-
18	cdb0eb19	real	644.3	644.3	194	24152	40	CROSS-HOST REPLICATION of captain-ahab 22bb52af (mglru, tier_threshold fixed at 3 = near-pure generational aging; 668.65 vs ref 656.5 on pc2
19	8a30f9dc	real ✗	crash	-	181	20758	42	eval: [FAIL] wall=41.98s error=benchmark[all] returned 1
20	67a47582	real	652.1	652.1	193	23854	41	CROSS-HOST REPLICATION (my final eval) of captain-ahab 69132a21 (mglru, tier protection disabled = pure generational aging; 686.07 on pc252,

All attempts on pc258 (davy-jones)

#	commit	class	score	tput	read MiB	refault s	wall s	design / title
1	5fd642b4	real =get_scan	518.7	518.7	137	11381	41	baseline: reference get_scan.c unchanged (scan-pid list + sampled LFU)
2	90cf3615	real ✗	crash	-	0	0	4	eval: [FAIL] wall=3.87s error=benchmark[all] returned 1
3	11b8cee0	real	625.1	625.1	163	16721	41	structural 1/3 (retry): scan-first CLOCK; state ARRAY index = hash(i_ino, index) since verifier forbids pointer shifts
4	378d80f2	real	641.7	641.7	164	17102	41	structural 2/3: scan-first freq-CLOCK: main list uses 2-bit freq counter (hit +1 cap 3, hand -1 & requeue, evict at 0), up to 4 hand passes;
5	8a3c64d0	real	630.8	630.8	161	16348	41	structural 3/3: freq-CLOCK + scan readahead protection: scan-list folios not yet read by scan thread (F_USED) get one requeue before evictio
6	fe0ca4fc	real	632.2	632.2	161	16473	41	freq-CLOCK (378d80f2) + ghost LRU_HASH(32k, key ino+index) of evicted main-list folios; ghost-hit refault admitted at freq 2 instead of 0. S
7	d9050830	real	605.2	605.2	175	19681	41	ablation of 378d80f2: pure freq-CLOCK, scan-thread folios go into main list like GET folios (no scan-first list). Tests whether scan-first h
8	372e2b1f	real	648.9	648.9	162	16572	41	378d80f2 + probation FIFO: non-scan folios enter new_list (F_NEW), move to main at freq 1 on first non-scan access; eviction order scan FIFO
9	fecab4a1	real	630.3	630.3	160	16281	41	372e2b1f + scan-thread accesses to main-list folios no longer bump freq (only GET hits count). Prev 648.9
10	1b4205bb	real	636.4	636.4	161	16295	41	372e2b1f + 3-bit freq counter (cap 7, was 3), up to 8 hand passes: stronger LFU memory for Zipfian GET pages under LevelDB's block cache. Pr
11	43fb724b	real	645.2	645.2	162	16673	41	REPLICATION of 372e2b1f (648.9), comment-only change: measuring noise; all my variants span 625-649
12	4989bfde	real	634.8	634.8	175	19410	41	CROSS-HOST REPLICATION of captain-ahab b87946a9 (656 on pc252) unchanged on pc258. My best here: 372e2b1f 648.9 / 645.3 replicate. Tests whe
13	ef943aac	real	646.0	646.0	164	16981	41	hybrid: 372e2b1f scan-first FIFO + probation FIFO kept, but folios earning a non-scan access are list_del'd to kernel reclaim (ahab's mechan
14	fd199fc3	real	634.9	634.9	163	16928	41	structural 1/3 on adaptive scan/GET partition: direction probe with FIXED scan floor 2048 pages (~17% of 47MiB cache) on 372e2b1f; scan FIFO
15	560c2235	real =noop	622.9	622.9	178	20095	41	CALIBRATION: exact seed noop.c (kernel default, never attaches) on MY host pc258. Needed to know if my best 372e2b1f (648.9/645.3) actually
16	043b62be	real	629.0	629.0	161	16361	40	372e2b1f with probation floor NEW_FLOOR 64 -> 512 (ahab's pc252 reserve curve: 32->627, 512->656/648, 2048->645). pc258 noop calibration = 6
17	479412c6	real =mglru	657.9	657.9	188	22912	41	CROSS-HOST: reference/mglru.c unchanged on pc258 (nemo: 657/669 on pc318 vs noop 616). pc258 noop=622.9, my best 372e2b1f=648.9/645.3
18	8b326f30	real	627.6	627.6	167	17663	41	structural 1/2: MGLRU (ref, 657.9 on pc258) + scan-first layer: scan_pids folios kept in a FIFO outside MGLRU and drained first on eviction;
19	6ca11c13	real =mglru	654.8	654.8	186	22406	40	REPLICATION #2: reference mglru.c unchanged on pc258 (1st: 657.94 479412c6; noop here 622.85). MGLRU+scan-first 8b326f30 = 627.65 => scan-fi
20	2e7dd61d	real	665.0	665.0	189	22974	40	FINAL eval, CROSS-HOST REPLICATION of captain-ahab 22bb52af (MGLRU, tier_threshold fixed at 3 = near-pure generational aging; 668.65 on pc25

All attempts on pc318 (captain-nemo)

#	commit	class	score	tput	read MiB	refaults	walls	design / title
1	0aab93ba	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
2	c5210e3c	real	581.1	581.1	165	17398	41	scan-first S3-FIFO (retry after compile fix: renamed need_more/ctx clashes). scan_pids pages in own FIFO drained first, floor 256 pages; GET
3	998b7146	real	608.3	608.3	180	20788	40	ablation of c5210e3c: drop scan-first list; scan-thread pages enter S3-FIFO small like others, scan accesses still ignored for freq. Tests w
4	e6c0f0c1	real	591.5	591.5	170	18591	41	structural 1/3 history-CLOCK: single CLOCK list, credits<=3; per-page GET freq in LRU hash keyed (inode,index), 4x cgroup pages, outlives ev
5	55919a84	real =noop	615.7	615.7	180	20418	41	calibration: exact seed noop.c (kernel default reclaim, no struct_ops attach) on pc318, to give my 581/608/591 attempts a same-host baseline
6	f3a2e20c	real	611.5	611.5	178	20368	41	S3-FIFO (998b7146, 608) + LevelDB table-tail protection: last 8 pages of each file (index/filter/footer) promoted small->main and skipped by
7	b3cf69a5	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
8	cd7ae862	real	603.1	603.1	180	20719	41	instrumentation rider retry (fix: skel-exported globals must be __u64, not u64). Policy = f3a2e20c S3-FIFO+tail-protect unchanged; counters
9	18e0c88e	real	596.7	596.7	167	17741	41	structural 1/3 consumed-scan-first (base f3a2e20c S3-FIFO+tail): scan_pids pages -> scan FIFO; evict first those the scanner already read (c
10	5b4d4022	real	591.7	591.7	160	16216	41	REPLICATION on pc318 of davy-jones 372e2b1f (648.9 on pc258, top raw score), code unchanged. Same-host noop here = 615.7; tells whether the
11	4094f127	real =mglru	657.4	657.4	196	24493	41	unexplored basin: reference mglru.c unchanged on pc318 (noop 615.7). Team evidence (ahab b87946a9, 2d090442) says scan pages are NOT junk an
12	35f7f9f5	real =mglru	669.5	669.5	192	23479	40	REPLICATION: reference mglru.c unchanged + 1 comment line (prev 657.4 on pc318 vs noop 615.7). Confirms or kills the +6.8% before tuning
13	33166ec4	real	631.3	631.3	185	21931	41	structural 1/3 MGLRU+refault activation: reference mglru.c (657.4/669.5 on pc318, noop 615.7) + kernel workingset rule: ghost stores evictio
14	45a24f02	real	657.9	657.9	202	25813	41	structural 2/3 on MGLRU: reference mglru.c + never-read reserve (newest 512 insertions with refs<=1 moved to next gen instead of evicted), g
15	15a0bf0e	real	649.4	649.4	192	23678	41	structural 3/3 on MGLRU: reference mglru.c + refs decay (protected folio refs-- on each promotion to next gen; refs never reset in reference
16	aa686e0d	real =mru	603.6	603.6	177	20174	41	unexplored extreme: reference mru.c unchanged on pc318. Motivation: ahab 22bb52af MGLRU tier_threshold=3 (no freq protection, head-insert =>
17	b9adb056	real	633.1	633.1	194	24164	40	CROSS-HOST REPLICATION of captain-ahab 22bb52af on pc318: reference mglru.c with fixed tier_threshold=MAX_NR_TIERS-1 (668.65 vs 656.5 defaul
18	8dffd87c	real	648.1	648.1	192	23784	41	MGLRU + bounded ghost: reference mglru.c with ghost map 400k per-CPU LRU -> 12288 common LRU (~cgroup pages), so PID tier controller counts
19	62948be4	real =mglru	647.6	647.6	192	23752	40	REPLICATION #3 on pc318: reference mglru.c unchanged (+comment). Prior 657.40/669.46; noop 615.74. Team consensus final policy (all tier/gho
20	a8531cd7	real =mglru	661.7	661.7	191	23369	41	FINAL eval: noop replicate #2 on pc318 (seed noop.c + comment) to tighten the baseline under the MGLRU claim (noop #1 615.74; MGLRU 657.40/6

pc252 (captain-ahab): best policy #70 69132a21 — score 686.1, 686.1 ops/s — overall best

Agent's title: MGLRU tier sweep, end point: tier protection DISABLED entirely (TIER_PROTECT 0: oldest-gen folios evicted regardless of refs; only locked/writeback/dirty skipped) = pure generational aging. Reflection on 22bb52af (threshold 3 = 668.65, my best, +4.3% vs same-host noop 641): threshold curve 0->639.0, PID->656.5, 3->668.7 is monotonic (less protection = better on pc252). This tests whether the trend continues to zero protection

Closest seed by line similarity: mglru 100%, sampling 34%, mru 33%. Lines that differ from mglru are highlighted.

```
// ===================================================================================
// SECTION: BPF KERNEL CODE
// ===================================================================================
// EVOLVE-BLOCK-START
// MGLRU: a BPF port of Linux's multi-generational LRU. Folios sit in generations (by age)
// and tiers (by access frequency); aging advances the generation counters and eviction
// takes from the oldest generation, with a ghost map feeding refault information back.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ARRAY_SIZE(x) (sizeof(x) / sizeof((x)[0]))
#define INT64_MAX (9223372036854775807LL)

// #define DEBUG
#ifdef DEBUG
#define dbg_printk(fmt, ...) bpf_printk(fmt, ##__VA_ARGS__)
#else
#define dbg_printk(fmt, ...)
#endif

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define atomic_long_read(ptr) __sync_fetch_and_add(ptr, 0)
#define atomic_long_zero(ptr) __sync_fetch_and_and(ptr, 0)
#define atomic_long_store(ptr, val) __sync_lock_test_and_set(ptr, val)

#define DEFINE_MIN_SEQ(lrugen) \
    unsigned long min_seq = READ_ONCE(lrugen->min_seq)
#define DEFINE_MAX_SEQ(lrugen) \
    unsigned long max_seq = READ_ONCE(lrugen->max_seq)

//////////
// Maps //
//////////

#define MAX_NR_FOLIOS 4000000
#define MAX_NR_GHOST_ENTRIES 400000

struct folio_metadata {
    s64 accesses;
    s64 gen;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, __u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, MAX_NR_FOLIOS);
} folio_metadata_map SEC(".maps");

//////////////////
// Ghost Enties //
//////////////////

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    __uint(max_entries, MAX_NR_GHOST_ENTRIES);
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

static inline void insert_ghost_entry_for_folio(struct folio *folio, int tier) {
    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 ghost_val = (u8) tier;
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
            bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline int folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 *tier;
    tier = bpf_map_lookup_elem(&ghost_map, &key);
    if (tier == NULL) {
        return -1;
    }
    // TODO: handle non-ENOENT errors
    bpf_map_delete_elem(&ghost_map, &key);
    return (int)(*tier);
}

////////////////////////////////////////////////////////////////////////////////////
//  _  ____ ___  _   _ ____ _____  _    _   _ _____ ____    ____ ___  ____  _____  //
// | |/ / ___/ _ \| \ | / ___|_   _|/ \  | \ | |_   _/ ___|  / ___/ _ \|  _ \| ____| //
// | ' / |  | | | |  \| \___ \ | | / _ \ |  \| | | | \___ \ | |  | | | | | | |  _|   //
// | . \ |__| |_| | |\  |___) || |/ ___ \| |\  | | |  ___) || |__| |_| | |_| | |___  //
// |_|\_\____\___/|_| \_|____/ |_/_/   \_\_| \_| |_| |____/  \____\___/|____/|_____| //
////////////////////////////////////////////////////////////////////////////////////

// Constants

#define MAX_NR_TIERS 4
#define TIER_PROTECT 0
#define MIN_NR_GENS 2
#define MAX_NR_GENS 4
#define NR_HIST_GENS 1
#define MIN_LRU_BATCH 64
// Global policy metadata
struct mglru_global_metadata {
    struct bpf_spin_lock lock;
    unsigned long max_seq;
    unsigned long min_seq;
    s64 evicted[MAX_NR_TIERS];
    s64 refaulted[MAX_NR_TIERS];
    s64 tier_selected[MAX_NR_TIERS];
    s64 success_evicted;
    s64 failed_evicted;
    unsigned long avg_refaulted[MAX_NR_TIERS];
    unsigned long avg_total[MAX_NR_TIERS];
    unsigned long protected[MAX_NR_TIERS - 1];
    long nr_pages[MAX_NR_GENS];
};

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, int);
    __type(value, struct mglru_global_metadata);
    __uint(max_entries, 1);
} mglru_global_metadata_map SEC(".maps");

#define DEFINE_LRUGEN_void                                                  \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                         \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return;                                                             \
    }
```

```
#define DEFINE_LRUGEN_int                                                   \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                         \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return -1;                                                          \
    }

#define DEFINE_LRUGEN_bool                                                  \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                         \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return false;                                                       \
    }

// TODO: Hook these up with the eventual policy
// Helpers to update metadata

#define assert_valid_tier_0(tier_idx)                                   \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {                 \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
               tier_idx);                                   \
        return;                                             \
    }

#define assert_valid_gen_0(gen_idx)                                         \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                            \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return;                                                             \
    }

#define assert_valid_tier_1(tier_idx)                                   \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {                 \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
               tier_idx);                                   \
        return -1;                                          \
    }

#define assert_valid_gen_1(gen_idx)                                         \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                            \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return -1;                                                          \
    }

inline void update_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                  s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->refaulted[tier_idx], delta);
}

inline void update_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->evicted[tier_idx], delta);
}

inline void update_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx,
                 s64 delta)
{
    assert_valid_gen_0(gen_idx);
    __sync_fetch_and_add(&lrugen->nr_pages[gen_idx], delta);
}

inline void update_tier_selected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                 s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->tier_selected[tier_idx], delta);
}


inline s64 read_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->refaulted[tier_idx]));
}

inline s64 read_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->evicted[tier_idx]));
}

inline s64 read_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx)
{
    assert_valid_gen_1(gen_idx);
    return max(0, atomic_long_read(&lrugen->nr_pages[gen_idx]));
}

// Invoke when promoting a folio in the eviction iteration
// See: https://github.com/cache-ext/linux-cachestream/blob/c22ffcac6b53ef4054483070fb902895ef10fd12/mm/vmscan.c#L4941-L4952
inline void update_protected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                  s64 delta)
{
    __sync_fetch_and_add(&lrugen->protected[tier_idx - 1], delta);
}

// Gen lists
static __u64 mglru_lists[MAX_NR_GENS];

/******************************************************************************
 *                           PID controller
 ******************************************************************************/

/*
 * A feedback loop based on Proportional-Integral-Derivative (PID) controller.
 *
 * The P term is refaulted/(evicted+protected) from a tier in the generation
 * currently being evicted; the I term is the exponential moving average of the
 * P term over the generations previously evicted, using the smoothing factor
 * 1/2; the D term isn't supported.
 *
 * The setpoint (SP) is always the first tier of one type; the process variable
 * (PV) is either any tier of the other type or any other tier of the same
 * type.
 *
 * The error is the difference between the SP and the PV; the correction is to
 * turn off protection when SP>PV or turn on protection when SP<PV.
 *
 * For future optimizations:
 * 1. The D term may discount the other two terms over time so that long-lived
 *    generations can resist stale information.
 */

 // Avoid name conflict with kernel struct ctrl_pos if MGLRU is enabled
 struct ctrl_pos___x {
    unsigned long refaulted;
    unsigned long total;
    int gain;
};

static inline void read_ctrl_pos(struct mglru_global_metadata *lrugen, int tier,
              int gain, struct ctrl_pos___x *pos)
{
    pos->refaulted = lrugen->avg_refaulted[tier] +
             __sync_fetch_and_add(&lrugen->refaulted[tier], 0);
    pos->total = lrugen->avg_total[tier] +
            __sync_fetch_and_add(&lrugen->evicted[tier], 0);
    if (tier)
        pos->total += lrugen->protected[tier - 1];
    pos->gain = gain;
}

// Needs the LRU lock. Why???
static inline void reset_ctrl_pos(struct mglru_global_metadata *lrugen, bool carryover)
{
    int tier;
    bool clear = carryover ? NR_HIST_GENS == 1 : NR_HIST_GENS > 1;

    // lockdep_assert_held(&lruvec->lru_lock);

    if (!carryover && !clear)
        return;

    for (tier = 0; tier < MAX_NR_TIERS; tier++) {
        if (carryover) {
            unsigned long sum;

            sum = lrugen->avg_refaulted[tier] +
                  atomic_long_read(&lrugen->refaulted[tier]);
            WRITE_ONCE(lrugen->avg_refaulted[tier], sum / 2);

            sum = lrugen->avg_total[tier] +
                  atomic_long_read(&lrugen->evicted[tier]);
```

```
            if (tier)
                sum += lrugen->protected[tier - 1];
            WRITE_ONCE(lrugen->avg_total[tier], sum / 2);
        }

        if (clear) {
            atomic_long_zero(&lrugen->refaulted[tier]);
            atomic_long_zero(&lrugen->evicted[tier]);
            if (tier)
                WRITE_ONCE(lrugen->protected[tier - 1], 0);
        }
    }
}

static inline bool positive_ctrl_err(struct ctrl_pos___x *sp, struct ctrl_pos___x *pv)
{
    /*
     * Return true if the PV has a limited number of refaults or a lower
     * refaulted/total than the SP.
     */
    return pv->refaulted < MIN_LRU_BATCH ||
           pv->refaulted * (sp->total + MIN_LRU_BATCH) * sp->gain <=
               (sp->refaulted + 1) * pv->total * pv->gain;
}

/////////////////////////////////////
// Utils that use the PID controller //
/////////////////////////////////////

static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
{
    int tier;
    struct ctrl_pos___x sp, pv;

    /*
     * To leave a margin for fluctuations, use a larger gain factor (1:2).
     * This value is chosen because any other tier would have at least twice
     * as many refaults as the firsfirst tier.
     */
    read_ctrl_pos(lrugen, 0, 1, &sp);
    for (tier = 1; tier < MAX_NR_TIERS; tier++) {
        read_ctrl_pos(lrugen, tier, 2, &pv);
        if (!positive_ctrl_err(&sp, &pv))
            break;
    }

    return tier - 1;
}

// static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
// {
//  // TODO: Swap this with the real implementation once we add refault tracking
//  // support.
//  return 0;
// }

static inline void folio_inc_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to inc refs but folio not found in map.\n");
        return;
    }
    __sync_fetch_and_add(&metadata->accesses, 1);
}

static inline int folio_lru_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata)
        return -1;

    return atomic_long_read(&metadata->accesses);
}

static inline unsigned int lru_gen_from_seq(unsigned long seq)
{
    return seq % MAX_NR_GENS;
}
```

```
static inline bool folio_test_active(struct folio *folio)
{
    return folio_lru_refs(folio) >= 2;
}

// Only counts up to 3.
static inline int order_base_2(int n)
{
    if (n <= 1) {
        return 0;
    } else if (n <= 3) {
        return 1;
    } else if (n <= 7) {
        return 2;
    } else if (n <= 8) {
        return 3;
    }
    return 3;
}

static inline int lru_tier_from_refs(int refs)
{
    /* see the comment in folio_lru_refs() */
    // kernel does some off-by-one calculation here, we remove the +1
    // return order_base_2(refs + 1);
    return order_base_2(refs);
}

static inline bool gen_within_limits(unsigned int gen)
{
    return 0 <= gen && gen <= MAX_NR_GENS;
}

static inline int get_nr_gens(struct mglru_global_metadata *lrugen)
{
    return lrugen->max_seq - lrugen->min_seq + 1;
}

static inline bool gen_almost_empty(struct mglru_global_metadata *lrugen,
                    int min_seq)
{
    int oldest_gen = lru_gen_from_seq(min_seq);
    int nr_folios = read_nr_pages_stat(lrugen, oldest_gen);
    int threshold = 4;
    return nr_folios <= threshold;
}

// TODO: This is supposed to run with a lock.
static inline bool lru_gen_add_folio(struct folio *folio)
{
    unsigned long seq;
    if (folio_test_unevictable(folio)) {
        bpf_printk("cache_ext: Unevictable folio\n");
        return false;
    }

    DEFINE_LRUGEN_bool;
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    /*
     * There are four common cases for this page:
     * 1. If it's hot, i.e., freshly faulted in, add it to the youngest
     *    generation, and it's protected over the rest below.
     * 2. If it can't be evicted immediately, i.e., a dirty page pending
     *    writeback, add it to the second youngest generation.
     * 3. If it should be evicted first, e.g., cold and clean from
     *    folio_rotate_reclaimable(), add it to the oldest generation.
     * 4. Everything else falls between 2 & 3 above and is added to the
     *    second oldest generation if it's considered inactive, or the
     *    oldest generation otherwise. See lru_gen_is_active().
     */
    if (folio_test_active(folio))
        seq = max_seq;
    else if ((folio_test_reclaim(folio) &&
          (folio_test_dirty(folio) || folio_test_writeback(folio))))
        seq = max_seq - 1;
    else if (min_seq + MIN_NR_GENS >= max_seq)
        seq = min_seq;
    else
        seq = min_seq + 1;

    unsigned int gen = lru_gen_from_seq(seq);
    if (!gen_within_limits(gen)) {
        bpf_printk(
            "cache_ext: Invalid gen %d (min_seq=%lu, max_seq=%lu)\n",
```

```
            gen, min_seq, max_seq);
        return false;
    }

    // Update policy metadata
    struct folio_metadata metadata = { .accesses = 1, .gen = gen };
    __u64 key = (__u64)folio;
    int ret = bpf_map_update_elem(&folio_metadata_map, &key, &metadata,
                      BPF_ANY);
    if (ret != 0) {
        bpf_printk("cache_ext: Failed to save folio metadata\n");
        return false;
    }
    update_nr_pages_stat(lrugen, gen, folio_nr_pages(folio));

    // Update refaulted stats
    ret = folio_in_ghost(folio);
    if (ret >= 0) {
        int tier = ret;
        update_refaulted_stat(lrugen, tier, 1);
    }

    // lru_gen_update_size(lruvec, folio, -1, gen);
    ret = bpf_cache_ext_list_add(mglru_lists[gen], folio);
    if (ret != 0) {
        bpf_printk(
            "cache_ext: Failed to add folio to mglru_lists[%d]\n",
            gen);
        return false;
    }

    return true;
}

static inline bool should_run_aging(struct mglru_global_metadata *lrugen,
                    unsigned long max_seq)
{
    unsigned int gen;
    unsigned long old = 0;
    unsigned long young = 0;
    unsigned long total = 0;
    DEFINE_MIN_SEQ(lrugen);

    /* whether this lruvec is completely out of cold folios */
    if (min_seq + MIN_NR_GENS > max_seq) {
        return true;
    }

    unsigned long seq;
    int max_iter =  min(MAX_NR_GENS, max_seq - min_seq);
    for (int i = 0; i < max_iter; i++) {
        seq = min_seq + i;
        unsigned long size = 0;

        gen = lru_gen_from_seq(seq);

        size += max(read_nr_pages_stat(lrugen, gen), 0L);

        total += size;
        if (seq == max_seq)
            young += size;
        else if (seq + MIN_NR_GENS == max_seq)
            old += size;
    }

    /*
     * The aging tries to be lazy to reduce the overhead, while the eviction
     * stalls when the number of generations reaches MIN_NR_GENS. Hence, the
     * ideal number of generations is MIN_NR_GENS+1.
     */
    if (min_seq + MIN_NR_GENS < max_seq)
        return false;

    /*
     * It's also ideal to spread pages out evenly, i.e., 1/(MIN_NR_GENS+1)
     * of the total number of pages for each generation. A reasonable range
     * for this average portion is [1/MIN_NR_GENS, 1/(MIN_NR_GENS+2)]. The
     * aging cares about the upper bound of hot pages, while the eviction
     * cares about the lower bound of cold pages.
     */
    if (young * MIN_NR_GENS > total)
        return true;
    if (old * (MIN_NR_GENS + 2) < total)
        return true;

    return false;
}

static inline bool try_to_inc_min_seq(struct mglru_global_metadata *lrugen)
{
    // TODO: Check if the min_seq list is empty and if it is, increase min_seq.
    DEFINE_MIN_SEQ(lrugen);
    if (!gen_almost_empty(lrugen, min_seq)) {
        return false;
    }
    WRITE_ONCE(lrugen->min_seq, lrugen->min_seq + 1);
    reset_ctrl_pos(lrugen, true);
    return true;
}

static inline bool try_to_inc_max_seq(struct mglru_global_metadata *lrugen)
{
    // int prev, next;
    // spin_lock_irq(&lruvec->lru_lock);
    if (get_nr_gens(lrugen) == MAX_NR_GENS) {
        // Try to increase min_seq
        int ret = try_to_inc_min_seq(lrugen);
        if (!ret) {
            return false;
        }
    }
    // spin_unlock_irq(&lruvec->lru_lock);

    // TODO: Do we need this?
    /*
     * Update the active/inactive LRU sizes for compatibility. Both sides of
     * the current max_seq need to be covered, since max_seq+1 can overlap
     * with min_seq[LRU_GEN_ANON] if swapping is constrained. And if they do
     * overlap, cold/hot inversion happens.
     */
    // prev = lru_gen_from_seq(lrugen->max_seq - 1);
    // next = lru_gen_from_seq(lrugen->max_seq + 1);

    // for (type = 0; type < ANON_AND_FILE; type++) {
    //  for (zone = 0; zone < MAX_NR_ZONES; zone++) {
    //      enum lru_list lru = type * LRU_INACTIVE_FILE;
    //      long delta = lrugen->nr_pages[prev] -
    //               lrugen->nr_pages[next];

    //      if (!delta)
    //          continue;

    //      __update_lru_size(lruvec, lru, zone, delta);
    //      __update_lru_size(lruvec, lru + LRU_ACTIVE, zone, -delta);
    //  }
    // }

    reset_ctrl_pos(lrugen, false);

    // We don't use the timestamp metadata for our MGLRU
    /* make sure preceding modifications appear */
    WRITE_ONCE(lrugen->max_seq, lrugen->max_seq + 1);
    return true;

    // spin_unlock_irq(&lruvec->lru_lock);
}

////////////////////////////////////////////////////////////
//  ____    _    ____ _   _ _____     _______ ____  ____  //
// |  _ \  / \  / ___| | | | ____|   | ____\ \/ /_ _|  _ \ //
// | |_) |/ _ \| |   | |_| |  _|     |  _|  \  / | | |_) |//
// |  __// ___ \ |___|  _  | |___    | |___ /  \ | |  __/ //
// |_|  /_/   \_\____|_| |_|_____|___|_____/_/\_\___|_|    //
//                              |_____|                   //
////////////////////////////////////////////////////////////


////////////
// Policy //
////////////


struct eviction_metadata {
    __u64 curr_gen;
    __u64 next_gen;
    __u64 iter_reached;
    __u64 tier_threshold;
};
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, u32);
```

```
    __type(value, struct eviction_metadata);
} mglru_percpu_array SEC(".maps");


struct eviction_metadata * get_eviction_metadata() {
    u32 key = 0;
    return bpf_map_lookup_elem(&mglru_percpu_array, &key);
}

void set_eviction_metadata(struct eviction_metadata *eviction_meta) {
    u32 key = 0;
    int ret = bpf_map_update_elem(&mglru_percpu_array, &key, eviction_meta, BPF_ANY);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to update eviction metadata\n");
    }
}

inline bool is_folio_relevant(struct folio *folio)
{
    if (!folio) {
        return false;
    }
    if (folio->mapping == NULL) {
        return false;
    }
    if (folio->mapping->host == NULL) {
        return false;
    }
    bool res = inode_in_watchlist(folio->mapping->host->i_ino);
    return res;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_int;
    WRITE_ONCE(lrugen->max_seq, MIN_NR_GENS + 1);
    for (int i = 0; i < MAX_NR_GENS; i++) {
        __u64 list_ptr = bpf_cache_ext_ds_registry_new_list(memcg);
        if (list_ptr == 0) {
                    bpf_printk(
            "cache_ext: Failed to allocate list for gen %d\n",
            i);
            return -1;
        }
        mglru_lists[i] = list_ptr;
    }
    return 0;
}


// MGLRU iteration function. Logic is mainly ported from sort_folio.
static int mglru_iter_fn(int idx, struct cache_ext_list_node *a)
{
    // We skip the following cases from sort_folio since we asssume we don't
    // accept these folios in this policy:
    // - Unevictable folios
    // - Swap-backed folios
    // - Promoted folios (these only appear through PTE accesses,
    //                    fd-accessed folios are promoted based on their tier)

    struct mglru_global_metadata *lrugen;
    int key__ = 0;
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);
    if (!lrugen) {
        bpf_printk(
            "cache_ext: Failed to lookup lrugen metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }

    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (!eviction_meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get eviction metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }
    eviction_meta->iter_reached = idx;

    // Get folio metadata
    __u64 key = (__u64)a->folio;
    struct folio_metadata *meta =
        bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get metadata\n");
        // TODO: Maybe we should evict it instead?
        return CACHE_EXT_EVICT_NODE;
    }
    int tier_threshold = eviction_meta->tier_threshold;
    if (tier_threshold > MAX_NR_TIERS || tier_threshold < 0) {
        bpf_printk("cache_ext: Invalid tier threshold %d\n", tier_threshold);
    }
    // int tier_threshold = 2;
    int tier = lru_tier_from_refs(atomic_long_read(&meta->accesses));

    /* protected (captain-ahab: tier protection disabled -> pure generational aging) */
    if (TIER_PROTECT && tier > tier_threshold) {
        update_protected_stat(lrugen, tier, folio_nr_pages(a->folio));
        // promote to next gen
        // TODO: Update nr_pages stats
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }

    /* waiting for writeback */
    if (folio_test_locked(a->folio) || folio_test_writeback(a->folio) ||
        folio_test_dirty(a->folio)) {
        // promote to next gen
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_void;

    bool inc_max_seq_failed = false;
    bpf_spin_lock(&lrugen->lock);
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    if (should_run_aging(lrugen, max_seq)) {
        if (!try_to_inc_max_seq(lrugen)){
            // Workaround: Not allowed to call bpf_printk under spinlock
            inc_max_seq_failed = true;
        }
    }
    if (max_seq - min_seq > MIN_NR_GENS)
        try_to_inc_min_seq(lrugen);
    // Read min/max seq again
    min_seq = READ_ONCE(lrugen->min_seq);
    max_seq = READ_ONCE(lrugen->max_seq);
    int oldest_gen = lru_gen_from_seq(min_seq);
    volatile unsigned int next_gen = (oldest_gen + 1) % MAX_NR_GENS;
    bpf_spin_unlock(&lrugen->lock);

    if (inc_max_seq_failed) {
        bpf_printk("cache_ext: Failed to increment max_seq\n");
    }

    // captain-ahab: fixed threshold instead of the PID controller: protect only folios with >=16 refs (tier 3)
    int tier_threshold = MAX_NR_TIERS - 1;
    update_tier_selected_stat(lrugen, tier_threshold, 1);

    // Save eviction metadata for stats
    struct eviction_metadata ev_meta = {
        .curr_gen = oldest_gen,
        .next_gen = next_gen,
        .tier_threshold = tier_threshold,
    };
    set_eviction_metadata(&ev_meta);

    assert_valid_gen_0(next_gen);

    __u64 next_gen_list = mglru_lists[next_gen];
    __u64 oldest_gen_list = mglru_lists[oldest_gen];
    struct cache_ext_iterate_opts opts = {
        .continue_list = next_gen_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };


    int ret = bpf_cache_ext_list_iterate_extended(
```

```
        memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to iterate list\n");
        return;
    }
    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (eviction_meta == NULL) return;
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        min_seq = READ_ONCE(lrugen->min_seq);
        oldest_gen = lru_gen_from_seq(min_seq);
        next_gen = (oldest_gen + 1) % MAX_NR_GENS;
        __u64 next_gen_list = mglru_lists[next_gen];
        __u64 oldest_gen_list = mglru_lists[oldest_gen];
        struct cache_ext_iterate_opts opts = {
            .continue_list = next_gen_list,
            .continue_mode = CACHE_EXT_ITERATE_TAIL,
            .evict_list = CACHE_EXT_ITERATE_SELF,
            .evict_mode = CACHE_EXT_ITERATE_TAIL,
        };
        int ret = bpf_cache_ext_list_iterate_extended(
            memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
        if (ret < 0) {
            bpf_printk("cache_ext: Failed to iterate list\n");
            return;
        }
    }
    s64 success_evicted = eviction_ctx->nr_folios_to_evict;
    s64 failed_evicted = max(0, eviction_ctx->request_nr_folios_to_evict - eviction_ctx->nr_folios_to_evict);
    __sync_fetch_and_add(&lrugen->failed_evicted, failed_evicted);
    __sync_fetch_and_add(&lrugen->success_evicted, success_evicted);
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        bpf_printk("cache_ext: Failed to evict requested number of folios: %d/%d. Used list idx %d, list ptr: %p. Iterated %d times.",
                eviction_ctx->nr_folios_to_evict,
                eviction_ctx->request_nr_folios_to_evict,
                oldest_gen,
                oldest_gen_list,
                eviction_meta->iter_reached);
    }
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    lru_gen_add_folio(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    folio_inc_refs(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    DEFINE_LRUGEN_void;
    // Remove tracked metadata
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to delete folio metadata but not found in map.\n");
        return;
    }
    // Add ghost entry for refault detection
    int tier = lru_tier_from_refs(atomic_long_read(&metadata->accesses));
    insert_ghost_entry_for_folio(folio, tier);

    // Update generation page count
    update_evicted_stat(lrugen, tier, 1);
    update_nr_pages_stat(lrugen, metadata->gen, -folio_nr_pages(folio));

    bpf_map_delete_elem(&folio_metadata_map, &key);
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the MGLRU policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "evo_policy.skel.h"
#include "dir_watcher.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";

struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = { { "watch_dir", 'w', "DIR", 0,
                      "Directory to watch" },
                    { "cgroup_path", 'c', "PATH", 0,
                      "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
                    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 } };

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static volatile sig_atomic_t exiting;

static void sig_handler(int signo)
{
    exiting = 1;
}

int main(int argc, char **argv)
{
    int ret = 1;
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    int cgroup_fd = -1;
    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    {
        struct sigaction sa;
        memset(&sa, 0, sizeof(sa));
        sigemptyset(&sa.sa_mask);
```

```
    sa.sa_handler = sig_handler;
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }
}

// Parse command line arguments
struct cmdline_args args = { 0 };
struct argp argp = { options, parse_opt, 0, 0 };
argp_parse(&argp, argc, argv, 0, 0, &args);

// Validate arguments
if (args.watch_dir == NULL) {
    fprintf(stderr, "Missing required argument: watch_dir\n");
    return 1;
}

if (args.cgroup_path == NULL) {
    fprintf(stderr, "Missing required argument: cgroup_path\n");
    return 1;
}

// Does watch_dir exist?
if (access(args.watch_dir, F_OK) == -1) {
    fprintf(stderr, "Directory does not exist: %s\n",
        args.watch_dir);
    return 1;
}

// Get full path of watch_dir
char watch_dir_full_path[PATH_MAX];
if (realpath(args.watch_dir, watch_dir_full_path) == NULL) {
    perror("realpath");
    return 1;
}

// TODO: Enable longer length
if (strlen(watch_dir_full_path) > 128) {
    fprintf(stderr, "watch_dir path too long\n");
    return 1;
}

// Open cgroup directory early
cgroup_fd = open(args.cgroup_path, O_RDONLY);
if (cgroup_fd < 0) {
    perror("Failed to open cgroup path");
    return 1;
}

// Open skel
skel = evo_policy_bpf__open();
if (skel == NULL) {
    perror("Failed to open BPF skeleton");
    goto cleanup;
}

// Set watch_dir
skel->rodata->watch_dir_path_len = strlen(watch_dir_full_path);
strcpy(skel->rodata->watch_dir_path, watch_dir_full_path);

// Load programs
ret = evo_policy_bpf__load(skel);
if (ret) {
    perror("Failed to load BPF skeleton");
    goto cleanup;
}
// Initialize inode_watchlist map
ret = initialize_watch_dir_map(args.watch_dir,
                  bpf_map__fd(skel->maps.inode_watchlist), false);

// Attach cache_ext_ops to the specific cgroup
link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
if (link == NULL) {
    perror("Failed to attach cache_ext_ops to cgroup");
    goto cleanup;
}

// Attach probes
ret = evo_policy_bpf__attach(skel);
if (ret) {
    perror("Failed to attach BPF programs");
    goto cleanup;
}

printf("evo_policy (MGLRU) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

pc257 (jack-sparrow): best policy #56 317ee824 — score 685.1, 685.1 ops/s

Agent's title: CROSS-HOST REPLICATION: reference/mglru.c unchanged (+1 comment) on pc257, per captain-nemo (657.4/669.5 on pc318 vs noop 615.7). My best on pc257 so far: 641.0 (junk-list+freq-history, small 50%).

Closest seed by line similarity: mglru 100%, sampling 34%, mru 33%. Lines that differ from mglru are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// MGLRU: a BPF port of Linux's multi-generational LRU. Folios sit in generations (by age)
// and tiers (by access frequency); aging advances the generation counters and eviction
// takes from the oldest generation, with a ghost map feeding refault information back.
// (cross-host replication of reference mglru.c by jack-sparrow on pc257)
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ARRAY_SIZE(x) (sizeof(x) / sizeof((x)[0]))
#define INT64_MAX (9223372036854775807LL)

// #define DEBUG
#ifdef DEBUG
#define dbg_printk(fmt, ...) bpf_printk(fmt, ##__VA_ARGS__)
#else
#define dbg_printk(fmt, ...)
#endif

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define atomic_long_read(ptr) __sync_fetch_and_add(ptr, 0)
#define atomic_long_zero(ptr) __sync_fetch_and_and(ptr, 0)
#define atomic_long_store(ptr, val) __sync_lock_test_and_set(ptr, val)

#define DEFINE_MIN_SEQ(lrugen) \
    unsigned long min_seq = READ_ONCE(lrugen->min_seq)
#define DEFINE_MAX_SEQ(lrugen) \
    unsigned long max_seq = READ_ONCE(lrugen->max_seq)

/////////// 
// Maps //
///////////

#define MAX_NR_FOLIOS 4000000
#define MAX_NR_GHOST_ENTRIES 400000

struct folio_metadata {
    s64 accesses;
    s64 gen;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, __u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, MAX_NR_FOLIOS);
} folio_metadata_map SEC(".maps");

//////////////////
// Ghost Enties //
//////////////////

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    __uint(max_entries, MAX_NR_GHOST_ENTRIES);
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

static inline void insert_ghost_entry_for_folio(struct folio *folio, int tier) {
    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 ghost_val = (u8) tier;
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
            bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline int folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 *tier;
    tier = bpf_map_lookup_elem(&ghost_map, &key);
    if (tier == NULL) {
        return -1;
    }
    // TODO: handle non-ENOENT errors
    bpf_map_delete_elem(&ghost_map, &key);
    return (int)(*tier);
}

////////////////////////////////////////////////////////////////////////////////
//  _____ _____ _____ _____ _____ _____ _____ _____ _____ _____ _____ _____  //
// |  |  |     |  _  |  |  |   __|   __|  |  |  _  |  |  |   __|   __|  _  | //
// |     |  |  |     |  |  |   __|__   |     |     |  |  |   __|__   |     | //
// |__|__|_____|__|__|_____|_____|_____|__|__|__|__|_____|_____|_____|__|__| //
////////////////////////////////////////////////////////////////////////////////

// Constants

#define MAX_NR_TIERS 4
#define MIN_NR_GENS 2
#define MAX_NR_GENS 4
#define NR_HIST_GENS 1
#define MIN_LRU_BATCH 64
// Global policy metadata
struct mglru_global_metadata {
    struct bpf_spin_lock lock;
    unsigned long max_seq;
    unsigned long min_seq;
    s64 evicted[MAX_NR_TIERS];
    s64 refaulted[MAX_NR_TIERS];
    s64 tier_selected[MAX_NR_TIERS];
    s64 success_evicted;
    s64 failed_evicted;
    unsigned long avg_refaulted[MAX_NR_TIERS];
    unsigned long avg_total[MAX_NR_TIERS];
    unsigned long protected[MAX_NR_TIERS - 1];
    long nr_pages[MAX_NR_GENS];
};

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, int);
    __type(value, struct mglru_global_metadata);
    __uint(max_entries, 1);
} mglru_global_metadata_map SEC(".maps");

#define DEFINE_LRUGEN_void                                                     \
    struct mglru_global_metadata *lrugen;                                      \
    int key__ = 0;                                                             \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);          \
    if (!lrugen) {                                                             \
        bpf_printk(                                                            \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return;                                                                \
    }

#define DEFINE_LRUGEN_int                                                      \
    struct mglru_global_metadata *lrugen;                                      \
    int key__ = 0;                                                             \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);          \
    if (!lrugen) {                                                             \
```

```
        bpf_printk(                                                 \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return -1;                                                  \
    }

#define DEFINE_LRUGEN_bool                                                  \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                 \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return false;                                               \
    }

// TODO: Hook these up with the eventual policy
// Helpers to update metadata

#define assert_valid_tier_0(tier_idx)                               \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {               \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
               tier_idx);                                   \
        return;                                                 \
    }

#define assert_valid_gen_0(gen_idx)                                         \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                            \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return;                                                 \
    }

#define assert_valid_tier_1(tier_idx)                               \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {               \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
               tier_idx);                                   \
        return -1;                                              \
    }

#define assert_valid_gen_1(gen_idx)                                         \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                            \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return -1;                                              \
    }

inline void update_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
              s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->refaulted[tier_idx], delta);
}

inline void update_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
             s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->evicted[tier_idx], delta);
}

inline void update_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx,
              s64 delta)
{
    assert_valid_gen_0(gen_idx);
    __sync_fetch_and_add(&lrugen->nr_pages[gen_idx], delta);
}

inline void update_tier_selected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
              s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->tier_selected[tier_idx], delta);
}


inline s64 read_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->refaulted[tier_idx]));
}

inline s64 read_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->evicted[tier_idx]));
}

inline s64 read_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx)
{
    assert_valid_gen_1(gen_idx);
    return max(0, atomic_long_read(&lrugen->nr_pages[gen_idx]));
}

// Invoke when promoting a folio in the eviction iteration
// See: https://github.com/cache-ext/linux-cachestream/blob/c22ffcac6b53ef4054483070fb902895ef10fd12/mm/vmscan.c#L4941-L4952
inline void update_protected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
              s64 delta)
{
    __sync_fetch_and_add(&lrugen->protected[tier_idx - 1], delta);
}

// Gen lists
static __u64 mglru_lists[MAX_NR_GENS];

/******************************************************************************
 *                           PID controller
 ******************************************************************************/

/*
 * A feedback loop based on Proportional-Integral-Derivative (PID) controller.
 *
 * The P term is refaulted/(evicted+protected) from a tier in the generation
 * currently being evicted; the I term is the exponential moving average of the
 * P term over the generations previously evicted, using the smoothing factor
 * 1/2; the D term isn't supported.
 *
 * The setpoint (SP) is always the first tier of one type; the process variable
 * (PV) is either any tier of the other type or any other tier of the same
 * type.
 *
 * The error is the difference between the SP and the PV; the correction is to
 * turn off protection when SP>PV or turn on protection when SP<PV.
 *
 * For future optimizations:
 * 1. The D term may discount the other two terms over time so that long-lived
 *    generations can resist stale information.
 */

 // Avoid name conflict with kernel struct ctrl_pos if MGLRU is enabled
 struct ctrl_pos___x {
    unsigned long refaulted;
    unsigned long total;
    int gain;
};

static inline void read_ctrl_pos(struct mglru_global_metadata *lrugen, int tier,
              int gain, struct ctrl_pos___x *pos)
{
    pos->refaulted = lrugen->avg_refaulted[tier] +
             __sync_fetch_and_add(&lrugen->refaulted[tier], 0);
    pos->total = lrugen->avg_total[tier] +
             __sync_fetch_and_add(&lrugen->evicted[tier], 0);
    if (tier)
        pos->total += lrugen->protected[tier - 1];
    pos->gain = gain;
}

// Needs the LRU lock. Why???
static inline void reset_ctrl_pos(struct mglru_global_metadata *lrugen, bool carryover)
{
    int tier;
    bool clear = carryover ? NR_HIST_GENS == 1 : NR_HIST_GENS > 1;

    // lockdep_assert_held(&lruvec->lru_lock);

    if (!carryover && !clear)
        return;

    for (tier = 0; tier < MAX_NR_TIERS; tier++) {
        if (carryover) {
            unsigned long sum;

            sum = lrugen->avg_refaulted[tier] +
                  atomic_long_read(&lrugen->refaulted[tier]);
            WRITE_ONCE(lrugen->avg_refaulted[tier], sum / 2);

            sum = lrugen->avg_total[tier] +
                  atomic_long_read(&lrugen->evicted[tier]);
            if (tier)
                sum += lrugen->protected[tier - 1];
            WRITE_ONCE(lrugen->avg_total[tier], sum / 2);
        }

        if (clear) {
```

```
            atomic_long_zero(&lrugen->refaulted[tier]);
            atomic_long_zero(&lrugen->evicted[tier]);
            if (tier)
                WRITE_ONCE(lrugen->protected[tier - 1], 0);
        }
    }
}

static inline bool positive_ctrl_err(struct ctrl_pos___x *sp, struct ctrl_pos___x *pv)
{
    /*
     * Return true if the PV has a limited number of refaults or a lower
     * refaulted/total than the SP.
     */
    return pv->refaulted < MIN_LRU_BATCH ||
           pv->refaulted * (sp->total + MIN_LRU_BATCH) * sp->gain <=
               (sp->refaulted + 1) * pv->total * pv->gain;
}

/////////////////////////////////////
// Utils that use the PID controller //
/////////////////////////////////////

static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
{
    int tier;
    struct ctrl_pos___x sp, pv;

    /*
     * To leave a margin for fluctuations, use a larger gain factor (1:2).
     * This value is chosen because any other tier would have at least twice
     * as many refaults as the firsfirst tier.
     */
    read_ctrl_pos(lrugen, 0, 1, &sp);
    for (tier = 1; tier < MAX_NR_TIERS; tier++) {
        read_ctrl_pos(lrugen, tier, 2, &pv);
        if (!positive_ctrl_err(&sp, &pv))
            break;
    }

    return tier - 1;
}

// static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
// {
//  // TODO: Swap this with the real implementation once we add refault tracking
//  // support.
//  return 0;
// }

static inline void folio_inc_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to inc refs but folio not found in map.\n");
        return;
    }
    __sync_fetch_and_add(&metadata->accesses, 1);
}

static inline int folio_lru_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata)
        return -1;

    return atomic_long_read(&metadata->accesses);
}

static inline unsigned int lru_gen_from_seq(unsigned long seq)
{
    return seq % MAX_NR_GENS;
}

static inline bool folio_test_active(struct folio *folio)
{
    return folio_lru_refs(folio) >= 2;
}
```

```
// Only counts up to 3.
static inline int order_base_2(int n)
{
    if (n <= 1) {
        return 0;
    } else if (n <= 3) {
        return 1;
    } else if (n <= 7) {
        return 2;
    } else if (n <= 8) {
        return 3;
    }
    return 3;
}

static inline int lru_tier_from_refs(int refs)
{
    /* see the comment in folio_lru_refs() */
    // kernel does some off-by-one calculation here, we remove the +1
    // return order_base_2(refs + 1);
    return order_base_2(refs);
}

static inline bool gen_within_limits(unsigned int gen)
{
    return 0 <= gen && gen <= MAX_NR_GENS;
}

static inline int get_nr_gens(struct mglru_global_metadata *lrugen)
{
    return lrugen->max_seq - lrugen->min_seq + 1;
}

static inline bool gen_almost_empty(struct mglru_global_metadata *lrugen,
                    int min_seq)
{
    int oldest_gen = lru_gen_from_seq(min_seq);
    int nr_folios = read_nr_pages_stat(lrugen, oldest_gen);
    int threshold = 4;
    return nr_folios <= threshold;
}

// TODO: This is supposed to run with a lock.
static inline bool lru_gen_add_folio(struct folio *folio)
{
    unsigned long seq;
    if (folio_test_unevictable(folio)) {
        bpf_printk("cache_ext: Unevictable folio\n");
        return false;
    }

    DEFINE_LRUGEN_bool;
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    /*
     * There are four common cases for this page:
     * 1. If it's hot, i.e., freshly faulted in, add it to the youngest
     *    generation, and it's protected over the rest below.
     * 2. If it can't be evicted immediately, i.e., a dirty page pending
     *    writeback, add it to the second youngest generation.
     * 3. If it should be evicted first, e.g., cold and clean from
     *    folio_rotate_reclaimable(), add it to the oldest generation.
     * 4. Everything else falls between 2 & 3 above and is added to the
     *    second oldest generation if it's considered inactive, or the
     *    oldest generation otherwise. See lru_gen_is_active().
     */
    if (folio_test_active(folio))
        seq = max_seq;
    else if ((folio_test_reclaim(folio) &&
          (folio_test_dirty(folio) || folio_test_writeback(folio))))
        seq = max_seq - 1;
    else if (min_seq + MIN_NR_GENS >= max_seq)
        seq = min_seq;
    else
        seq = min_seq + 1;

    unsigned int gen = lru_gen_from_seq(seq);
    if (!gen_within_limits(gen)) {
        bpf_printk(
            "cache_ext: Invalid gen %d (min_seq=%lu, max_seq=%lu)\n",
            gen, min_seq, max_seq);
        return false;
    }

    // Update policy metadata
    struct folio_metadata metadata = { .accesses = 1, .gen = gen };
```

```
    __u64 key = (__u64)folio;
    int ret = bpf_map_update_elem(&folio_metadata_map, &key, &metadata,
                      BPF_ANY);
    if (ret != 0) {
        bpf_printk("cache_ext: Failed to save folio metadata\n");
        return false;
    }
    update_nr_pages_stat(lrugen, gen, folio_nr_pages(folio));

    // Update refaulted stats
    ret = folio_in_ghost(folio);
    if (ret >= 0) {
        int tier = ret;
        update_refaulted_stat(lrugen, tier, 1);
    }

    // lru_gen_update_size(lruvec, folio, -1, gen);
    ret = bpf_cache_ext_list_add(mglru_lists[gen], folio);
    if (ret != 0) {
        bpf_printk(
            "cache_ext: Failed to add folio to mglru_lists[%d]\n",
            gen);
        return false;
    }

    return true;
}

static inline bool should_run_aging(struct mglru_global_metadata *lrugen,
                    unsigned long max_seq)
{
    unsigned int gen;
    unsigned long old = 0;
    unsigned long young = 0;
    unsigned long total = 0;
    DEFINE_MIN_SEQ(lrugen);

    /* whether this lruvec is completely out of cold folios */
    if (min_seq + MIN_NR_GENS > max_seq) {
        return true;
    }

    unsigned long seq;
    int max_iter =  min(MAX_NR_GENS, max_seq - min_seq);
    for (int i = 0; i < max_iter; i++) {
        seq = min_seq + i;
        unsigned long size = 0;

        gen = lru_gen_from_seq(seq);

        size += max(read_nr_pages_stat(lrugen, gen), 0L);

        total += size;
        if (seq == max_seq)
            young += size;
        else if (seq + MIN_NR_GENS == max_seq)
            old += size;
    }

    /*
     * The aging tries to be lazy to reduce the overhead, while the eviction
     * stalls when the number of generations reaches MIN_NR_GENS. Hence, the
     * ideal number of generations is MIN_NR_GENS+1.
     */
    if (min_seq + MIN_NR_GENS < max_seq)
        return false;

    /*
     * It's also ideal to spread pages out evenly, i.e., 1/(MIN_NR_GENS+1)
     * of the total number of pages for each generation. A reasonable range
     * for this average portion is [1/MIN_NR_GENS, 1/(MIN_NR_GENS+2)]. The
     * aging cares about the upper bound of hot pages, while the eviction
     * cares about the lower bound of cold pages.
     */
    if (young * MIN_NR_GENS > total)
        return true;
    if (old * (MIN_NR_GENS + 2) < total)
        return true;

    return false;
}

static inline bool try_to_inc_min_seq(struct mglru_global_metadata *lrugen)
{
    // TODO: Check if the min_seq list is empty and if it is, increase min_seq.
    DEFINE_MIN_SEQ(lrugen);

    if (!gen_almost_empty(lrugen, min_seq)) {
        return false;
    }
    WRITE_ONCE(lrugen->min_seq, lrugen->min_seq + 1);
    reset_ctrl_pos(lrugen, true);
    return true;
}

static inline bool try_to_inc_max_seq(struct mglru_global_metadata *lrugen)
{
    // int prev, next;
    // spin_lock_irq(&lruvec->lru_lock);
    if (get_nr_gens(lrugen) == MAX_NR_GENS) {
        // Try to increase min_seq
        int ret = try_to_inc_min_seq(lrugen);
        if (!ret) {
            return false;
        }
    }
    // spin_unlock_irq(&lruvec->lru_lock);

    // TODO: Do we need this?
    /*
     * Update the active/inactive LRU sizes for compatibility. Both sides of
     * the current max_seq need to be covered, since max_seq+1 can overlap
     * with min_seq[LRU_GEN_ANON] if swapping is constrained. And if they do
     * overlap, cold/hot inversion happens.
     */
    // prev = lru_gen_from_seq(lrugen->max_seq - 1);
    // next = lru_gen_from_seq(lrugen->max_seq + 1);

    // for (type = 0; type < ANON_AND_FILE; type++) {
    //  for (zone = 0; zone < MAX_NR_ZONES; zone++) {
    //      enum lru_list lru = type * LRU_INACTIVE_FILE;
    //      long delta = lrugen->nr_pages[prev] -
    //               lrugen->nr_pages[next];

    //      if (!delta)
    //          continue;

    //      __update_lru_size(lruvec, lru, zone, delta);
    //      __update_lru_size(lruvec, lru + LRU_ACTIVE, zone, -delta);
    //  }
    // }

    reset_ctrl_pos(lrugen, false);

    // We don't use the timestamp metadata for our MGLRU
    /* make sure preceding modifications appear */
    WRITE_ONCE(lrugen->max_seq, lrugen->max_seq + 1);
    return true;

    // spin_unlock_irq(&lruvec->lru_lock);
}

//////////////////////////////////////////////////////////
//  _____   _   ____ _   _ _____   _____ _____ _____   //
//  / ___|  / \  / ___| | | | ____|   | ____\ \/ /_ _|  //
// | |     / _ \| |   | |_| |  _|     |  _|  \  / | |   //
// | |___ / ___ \ |___|  _  | |___    | |___ /  \ | |   //
//  \____/_/   \_\____|_| |_|_____|___|_____/_/\_\___|   //
//                          |_____|                     //
//////////////////////////////////////////////////////////


////////////
// Policy //
////////////


struct eviction_metadata {
    __u64 curr_gen;
    __u64 next_gen;
    __u64 iter_reached;
    __u64 tier_threshold;
};
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, u32);
    __type(value, struct eviction_metadata);
} mglru_percpu_array SEC(".maps");


struct eviction_metadata * get_eviction_metadata() {
    u32 key = 0;
```

```
    return bpf_map_lookup_elem(&mglru_percpu_array, &key);
}

void set_eviction_metadata(struct eviction_metadata *eviction_meta) {
    u32 key = 0;
    int ret = bpf_map_update_elem(&mglru_percpu_array, &key, eviction_meta, BPF_ANY);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to update eviction metadata\n");
    }
}

inline bool is_folio_relevant(struct folio *folio)
{
    if (!folio) {
        return false;
    }
    if (folio->mapping == NULL) {
        return false;
    }
    if (folio->mapping->host == NULL) {
        return false;
    }
    bool res = inode_in_watchlist(folio->mapping->host->i_ino);
    return res;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_int;
    WRITE_ONCE(lrugen->max_seq, MIN_NR_GENS + 1);
    for (int i = 0; i < MAX_NR_GENS; i++) {
        __u64 list_ptr = bpf_cache_ext_ds_registry_new_list(memcg);
        if (list_ptr == 0) {
                    bpf_printk(
            "cache_ext: Failed to allocate list for gen %d\n",
            i);
            return -1;
        }
        mglru_lists[i] = list_ptr;
    }
    return 0;
}


// MGLRU iteration function. Logic is mainly ported from sort_folio.
static int mglru_iter_fn(int idx, struct cache_ext_list_node *a)
{
    // We skip the following cases from sort_folio since we asssume we don't
    // accept these folios in this policy:
    // - Unevictable folios
    // - Swap-backed folios
    // - Promoted folios (these only appear through PTE accesses,
    //                    fd-accessed folios are promoted based on their tier)

    struct mglru_global_metadata *lrugen;
    int key__ = 0;
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);
    if (!lrugen) {
        bpf_printk(
            "cache_ext: Failed to lookup lrugen metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }

    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (!eviction_meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get eviction metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }
    eviction_meta->iter_reached = idx;

    // Get folio metadata
    __u64 key = (__u64)a->folio;
    struct folio_metadata *meta =
        bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get metadata\n");
        // TODO: Maybe we should evict it instead?
        return CACHE_EXT_EVICT_NODE;
    }

    int tier_threshold = eviction_meta->tier_threshold;
    if (tier_threshold > MAX_NR_TIERS || tier_threshold < 0) {
        bpf_printk("cache_ext: Invalid tier threshold %d\n", tier_threshold);
    }
    // int tier_threshold = 2;
    int tier = lru_tier_from_refs(atomic_long_read(&meta->accesses));

    /* protected */
    if (tier > tier_threshold) {
        update_protected_stat(lrugen, tier, folio_nr_pages(a->folio));
        // promote to next gen
        // TODO: Update nr_pages stats
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }

    /* waiting for writeback */
    if (folio_test_locked(a->folio) || folio_test_writeback(a->folio) ||
        folio_test_dirty(a->folio)) {
        // promote to next gen
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_void;

    bool inc_max_seq_failed = false;
    bpf_spin_lock(&lrugen->lock);
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    if (should_run_aging(lrugen, max_seq)) {
        if (!try_to_inc_max_seq(lrugen)){
            // Workaround: Not allowed to call bpf_printk under spinlock
            inc_max_seq_failed = true;
        }
    }
    if (max_seq - min_seq > MIN_NR_GENS)
        try_to_inc_min_seq(lrugen);
    // Read min/max seq again
    min_seq = READ_ONCE(lrugen->min_seq);
    max_seq = READ_ONCE(lrugen->max_seq);
    int oldest_gen = lru_gen_from_seq(min_seq);
    volatile unsigned int next_gen = (oldest_gen + 1) % MAX_NR_GENS;
    bpf_spin_unlock(&lrugen->lock);

    if (inc_max_seq_failed) {
        bpf_printk("cache_ext: Failed to increment max_seq\n");
    }

    int tier_threshold = get_tier_idx(lrugen);
    update_tier_selected_stat(lrugen, tier_threshold, 1);

    // Save eviction metadata for stats
    struct eviction_metadata ev_meta = {
        .curr_gen = oldest_gen,
        .next_gen = next_gen,
        .tier_threshold = tier_threshold,
    };
    set_eviction_metadata(&ev_meta);

    assert_valid_gen_0(next_gen);

    __u64 next_gen_list = mglru_lists[next_gen];
    __u64 oldest_gen_list = mglru_lists[oldest_gen];
    struct cache_ext_iterate_opts opts = {
        .continue_list = next_gen_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };


    int ret = bpf_cache_ext_list_iterate_extended(
        memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to iterate list\n");
        return;
    }
    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (eviction_meta == NULL) return;
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
```

```
        min_seq = READ_ONCE(lrugen->min_seq);
        oldest_gen = lru_gen_from_seq(min_seq);
        next_gen = (oldest_gen + 1) % MAX_NR_GENS;
        __u64 next_gen_list = mglru_lists[next_gen];
        __u64 oldest_gen_list = mglru_lists[oldest_gen];
        struct cache_ext_iterate_opts opts = {
            .continue_list = next_gen_list,
            .continue_mode = CACHE_EXT_ITERATE_TAIL,
            .evict_list = CACHE_EXT_ITERATE_SELF,
            .evict_mode = CACHE_EXT_ITERATE_TAIL,
        };
        int ret = bpf_cache_ext_list_iterate_extended(
            memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
        if (ret < 0) {
            bpf_printk("cache_ext: Failed to iterate list\n");
            return;
        }
    }
    s64 success_evicted = eviction_ctx->nr_folios_to_evict;
    s64 failed_evicted = max(0, eviction_ctx->request_nr_folios_to_evict - eviction_ctx->nr_folios_to_evict);
    __sync_fetch_and_add(&lrugen->failed_evicted, failed_evicted);
    __sync_fetch_and_add(&lrugen->success_evicted, success_evicted);
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        bpf_printk("cache_ext: Failed to evict requested number of folios: %d/%d. Used list idx %d, list ptr: %p. Iterated %d in a
                eviction_ctx->nr_folios_to_evict,
                eviction_ctx->request_nr_folios_to_evict,
                oldest_gen,
                oldest_gen_list,
                eviction_meta->iter_reached);
    }
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    lru_gen_add_folio(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    folio_inc_refs(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    DEFINE_LRUGEN_void;
    // Remove tracked metadata
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to delete folio metadata but not found in map.\n");
        return;
    }
    // Add ghost entry for refault detection
    int tier = lru_tier_from_refs(atomic_long_read(&metadata->accesses));
    insert_ghost_entry_for_folio(folio, tier);

    // Update generation page count
    update_evicted_stat(lrugen, tier, 1);
    update_nr_pages_stat(lrugen, metadata->gen, -folio_nr_pages(folio));

    bpf_map_delete_elem(&folio_metadata_map, &key);
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
```

```
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the MGLRU policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "evo_policy.skel.h"
#include "dir_watcher.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = { { "watch_dir", 'w', "DIR", 0,
                    "Directory to watch" },
                  { "cgroup_path", 'c', "PATH", 0,
                    "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
                  { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                  { 0 } };

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static volatile sig_atomic_t exiting;

static void sig_handler(int signo)
{
    exiting = 1;
}

int main(int argc, char **argv)
{
    int ret = 1;
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    int cgroup_fd = -1;
    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    {
        struct sigaction sa;
        memset(&sa, 0, sizeof(sa));
        sigemptyset(&sa.sa_mask);
        sa.sa_handler = sig_handler;
        if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
            perror("Failed to set up signal handling");
            return 1;
        }
    }

    // Parse command line arguments
```

```
    struct cmdline_args args = { 0 };
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, &args);

    // Validate arguments
    if (args.watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args.cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    // Does watch_dir exist?
    if (access(args.watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n",
            args.watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    char watch_dir_full_path[PATH_MAX];
    if (realpath(args.watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // TODO: Enable longer length
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    // Open skel
    skel = evo_policy_bpf__open();
    if (skel == NULL) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set watch_dir
    skel->rodata->watch_dir_path_len = strlen(watch_dir_full_path);
    strcpy(skel->rodata->watch_dir_path, watch_dir_full_path);

    // Load programs
    ret = evo_policy_bpf__load(skel);
    if (ret) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }

    // Initialize inode_watchlist map
    ret = initialize_watch_dir_map(args.watch_dir,
                        bpf_map__fd(skel->maps.inode_watchlist), false);

    // Attach cache_ext_ops to the specific cgroup
    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // Attach probes
    ret = evo_policy_bpf__attach(skel);
    if (ret) {
        perror("Failed to attach BPF programs");
        goto cleanup;
    }

    printf("evo_policy (MGLRU) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

pc258 (davy-jones): best policy #72 2e7dd61d — score 665.0, 665.0 ops/s

Agent's title: FINAL eval, CROSS-HOST REPLICATION of captain-ahab 22bb52af (MGLRU, tier_threshold fixed at 3 = near-pure generational aging; 668.65 on pc252 vs unchanged MGLRU 656.5 there) on pc258. pc258 baselines: noop 622.85, MGLRU unchanged 657.94/654.84, my freq-CLOCK+probation 648.9/645.3

Closest seed by line similarity: mglru 100%, sampling 34%, mru 33%. Lines that differ from mglru are highlighted.

```
// ===================================================================
// SECTION: BPF KERNEL CODE
// ===================================================================
// EVOLVE-BLOCK-START
// MGLRU: a BPF port of Linux's multi-generational LRU. Folios sit in generations (by age)
// and tiers (by access frequency); aging advances the generation counters and eviction
// takes from the oldest generation, with a ghost map feeding refault information back.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

// davy-jones: cross-host replication of captain-ahab 22bb52af on pc258
char _license[] SEC("license") = "GPL";

#define ARRAY_SIZE(x) (sizeof(x) / sizeof((x)[0]))
#define INT64_MAX (9223372036854775807LL)

// #define DEBUG
#ifdef DEBUG
#define dbg_printk(fmt, ...) bpf_printk(fmt, ##__VA_ARGS__)
#else
#define dbg_printk(fmt, ...)
#endif

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define atomic_long_read(ptr) __sync_fetch_and_add(ptr, 0)
#define atomic_long_zero(ptr) __sync_fetch_and_and(ptr, 0)
#define atomic_long_store(ptr, val) __sync_lock_test_and_set(ptr, val)

#define DEFINE_MIN_SEQ(lrugen) \
    unsigned long min_seq = READ_ONCE(lrugen->min_seq)
#define DEFINE_MAX_SEQ(lrugen) \
    unsigned long max_seq = READ_ONCE(lrugen->max_seq)

//////////
// Maps //
//////////

#define MAX_NR_FOLIOS 4000000
#define MAX_NR_GHOST_ENTRIES 400000

struct folio_metadata {
    s64 accesses;
    s64 gen;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, __u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, MAX_NR_FOLIOS);
} folio_metadata_map SEC(".maps");

//////////////////
// Ghost Enties //
//////////////////

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    __uint(max_entries, MAX_NR_GHOST_ENTRIES);
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

static inline void insert_ghost_entry_for_folio(struct folio *folio, int tier) {
    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 ghost_val = (u8) tier;
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
            bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline int folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 *tier;
    tier = bpf_map_lookup_elem(&ghost_map, &key);
    if (tier == NULL) {
        return -1;
    }
    // TODO: handle non-ENOENT errors
    bpf_map_delete_elem(&ghost_map, &key);
    return (int)(*tier);
}

////////////////////////////////////////////////////////////////////////////////////////
//  _  _______ ____  _  _____ _       ___ __  __ ____    ___  ____ _____ ____    //
// | |/ / ____|  _ \| | | ____| |     |_ _|  \/  |  _ \ / _ \|  _ \_   _/ ___|  //
// | ' /|  _| | |_) |  \| |  _| | |      | || |\/| | |_) | | | | |_) || | \___ \  //
// | . \| |___|  _ <| |\  | |___| |___   | || |  | |  __/| |_| |  _ < | |  ___) | //
// |_|\_\_____|_| \_\_| \_|_____|_____| |___|_|  |_|_|    \___/|_| \_\|_| |____/  //
////////////////////////////////////////////////////////////////////////////////////////

// Constants

#define MAX_NR_TIERS 4
#define MIN_NR_GENS 2
#define MAX_NR_GENS 4
#define NR_HIST_GENS 1
#define MIN_LRU_BATCH 64
// Global policy metadata
struct mglru_global_metadata {
    struct bpf_spin_lock lock;
    unsigned long max_seq;
    unsigned long min_seq;
    s64 evicted[MAX_NR_TIERS];
    s64 refaulted[MAX_NR_TIERS];
    s64 tier_selected[MAX_NR_TIERS];
    s64 success_evicted;
    s64 failed_evicted;
    unsigned long avg_refaulted[MAX_NR_TIERS];
    unsigned long avg_total[MAX_NR_TIERS];
    unsigned long protected[MAX_NR_TIERS - 1];
    long nr_pages[MAX_NR_GENS];
};

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, int);
    __type(value, struct mglru_global_metadata);
    __uint(max_entries, 1);
} mglru_global_metadata_map SEC(".maps");

#define DEFINE_LRUGEN_void                                                      \
    struct mglru_global_metadata *lrugen;                                       \
    int key__ = 0;                                                              \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);           \
    if (!lrugen) {                                                              \
        bpf_printk(                                                             \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return;                                                                 \
    }

#define DEFINE_LRUGEN_int                                                       \
    struct mglru_global_metadata *lrugen;                                       \
    int key__ = 0;                                                              \
```

```
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);      \
    if (!lrugen) {                                                         \
        bpf_printk(                                                        \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return -1;                                                         \
    }

#define DEFINE_LRUGEN_bool                                                     \
    struct mglru_global_metadata *lrugen;                                  \
    int key__ = 0;                                                         \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);      \
    if (!lrugen) {                                                         \
        bpf_printk(                                                        \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return false;                                                      \
    }

// TODO: Hook these up with the eventual policy
// Helpers to update metadata

#define assert_valid_tier_0(tier_idx)                                  \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {                \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
               tier_idx);                                          \
        return;                                                    \
    }

#define assert_valid_gen_0(gen_idx)                                            \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                           \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return;                                                            \
    }

#define assert_valid_tier_1(tier_idx)                                  \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {                \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
               tier_idx);                                          \
        return -1;                                                 \
    }

#define assert_valid_gen_1(gen_idx)                                            \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                           \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return -1;                                                         \
    }

inline void update_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                  s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->refaulted[tier_idx], delta);
}

inline void update_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->evicted[tier_idx], delta);
}

inline void update_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx,
                 s64 delta)
{
    assert_valid_gen_0(gen_idx);
    __sync_fetch_and_add(&lrugen->nr_pages[gen_idx], delta);
}

inline void update_tier_selected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                 s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->tier_selected[tier_idx], delta);
}


inline s64 read_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->refaulted[tier_idx]));
}

inline s64 read_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->evicted[tier_idx]));
}
```

```
inline s64 read_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx)
{
    assert_valid_gen_1(gen_idx);
    return max(0, atomic_long_read(&lrugen->nr_pages[gen_idx]));
}

// Invoke when promoting a folio in the eviction iteration
// See: https://github.com/cache-ext/linux-cachestream/blob/c22ffcac6b53ef4054483070fb902895ef10fd12/mm/vmscan.c#L4941-L4952
inline void update_protected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                  s64 delta)
{
    __sync_fetch_and_add(&lrugen->protected[tier_idx - 1], delta);
}

// Gen lists
static __u64 mglru_lists[MAX_NR_GENS];

/******************************************************************************
 *                           PID controller
 ******************************************************************************/

/*
 * A feedback loop based on Proportional-Integral-Derivative (PID) controller.
 *
 * The P term is refaulted/(evicted+protected) from a tier in the generation
 * currently being evicted; the I term is the exponential moving average of the
 * P term over the generations previously evicted, using the smoothing factor
 * 1/2; the D term isn't supported.
 *
 * The setpoint (SP) is always the first tier of one type; the process variable
 * (PV) is either any tier of the other type or any other tier of the same
 * type.
 *
 * The error is the difference between the SP and the PV; the correction is to
 * turn off protection when SP>PV or turn on protection when SP<PV.
 *
 * For future optimizations:
 * 1. The D term may discount the other two terms over time so that long-lived
 *    generations can resist stale information.
 */

 // Avoid name conflict with kernel struct ctrl_pos if MGLRU is enabled
 struct ctrl_pos___x {
    unsigned long refaulted;
    unsigned long total;
    int gain;
};

static inline void read_ctrl_pos(struct mglru_global_metadata *lrugen, int tier,
             int gain, struct ctrl_pos___x *pos)
{
    pos->refaulted = lrugen->avg_refaulted[tier] +
             __sync_fetch_and_add(&lrugen->refaulted[tier], 0);
    pos->total = lrugen->avg_total[tier] +
             __sync_fetch_and_add(&lrugen->evicted[tier], 0);
    if (tier)
        pos->total += lrugen->protected[tier - 1];
    pos->gain = gain;
}

// Needs the LRU lock. Why???
static inline void reset_ctrl_pos(struct mglru_global_metadata *lrugen, bool carryover)
{
    int tier;
    bool clear = carryover ? NR_HIST_GENS == 1 : NR_HIST_GENS > 1;

    // lockdep_assert_held(&lruvec->lru_lock);

    if (!carryover && !clear)
        return;

    for (tier = 0; tier < MAX_NR_TIERS; tier++) {
        if (carryover) {
            unsigned long sum;

            sum = lrugen->avg_refaulted[tier] +
                  atomic_long_read(&lrugen->refaulted[tier]);
            WRITE_ONCE(lrugen->avg_refaulted[tier], sum / 2);

            sum = lrugen->avg_total[tier] +
                  atomic_long_read(&lrugen->evicted[tier]);
            if (tier)
                sum += lrugen->protected[tier - 1];
            WRITE_ONCE(lrugen->avg_total[tier], sum / 2);
        }
```

```
        if (clear) {
            atomic_long_zero(&lrugen->refaulted[tier]);
            atomic_long_zero(&lrugen->evicted[tier]);
            if (tier)
                WRITE_ONCE(lrugen->protected[tier - 1], 0);
        }
    }
}

static inline bool positive_ctrl_err(struct ctrl_pos___x *sp, struct ctrl_pos___x *pv)
{
    /*
     * Return true if the PV has a limited number of refaults or a lower
     * refaulted/total than the SP.
     */
    return pv->refaulted < MIN_LRU_BATCH ||
           pv->refaulted * (sp->total + MIN_LRU_BATCH) * sp->gain <=
               (sp->refaulted + 1) * pv->total * pv->gain;
}

////////////////////////////////////
// Utils that use the PID controller //
////////////////////////////////////

static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
{
    int tier;
    struct ctrl_pos___x sp, pv;

    /*
     * To leave a margin for fluctuations, use a larger gain factor (1:2).
     * This value is chosen because any other tier would have at least twice
     * as many refaults as the firsfirst tier.
     */
    read_ctrl_pos(lrugen, 0, 1, &sp);
    for (tier = 1; tier < MAX_NR_TIERS; tier++) {
        read_ctrl_pos(lrugen, tier, 2, &pv);
        if (!positive_ctrl_err(&sp, &pv))
            break;
    }

    return tier - 1;
}

// static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
// {
//  // TODO: Swap this with the real implementation once we add refault tracking
//  // support.
//  return 0;
// }

static inline void folio_inc_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to inc refs but folio not found in map.\n");
        return;
    }
    __sync_fetch_and_add(&metadata->accesses, 1);
}

static inline int folio_lru_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata)
        return -1;

    return atomic_long_read(&metadata->accesses);
}

static inline unsigned int lru_gen_from_seq(unsigned long seq)
{
    return seq % MAX_NR_GENS;
}

static inline bool folio_test_active(struct folio *folio)
{
    return folio_lru_refs(folio) >= 2;
}

}

// Only counts up to 3.
static inline int order_base_2(int n)
{
    if (n <= 1) {
        return 0;
    } else if (n <= 3) {
        return 1;
    } else if (n <= 7) {
        return 2;
    } else if (n <= 8) {
        return 3;
    }
    return 3;
}

static inline int lru_tier_from_refs(int refs)
{
    /* see the comment in folio_lru_refs() */
    // kernel does some off-by-one calculation here, we remove the +1
    // return order_base_2(refs + 1);
    return order_base_2(refs);
}

static inline bool gen_within_limits(unsigned int gen)
{
    return 0 <= gen && gen <= MAX_NR_GENS;
}

static inline int get_nr_gens(struct mglru_global_metadata *lrugen)
{
    return lrugen->max_seq - lrugen->min_seq + 1;
}

static inline bool gen_almost_empty(struct mglru_global_metadata *lrugen,
                    int min_seq)
{
    int oldest_gen = lru_gen_from_seq(min_seq);
    int nr_folios = read_nr_pages_stat(lrugen, oldest_gen);
    int threshold = 4;
    return nr_folios <= threshold;
}

// TODO: This is supposed to run with a lock.
static inline bool lru_gen_add_folio(struct folio *folio)
{
    unsigned long seq;
    if (folio_test_unevictable(folio)) {
        bpf_printk("cache_ext: Unevictable folio\n");
        return false;
    }

    DEFINE_LRUGEN_bool;
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    /*
     * There are four common cases for this page:
     * 1. If it's hot, i.e., freshly faulted in, add it to the youngest
     *    generation, and it's protected over the rest below.
     * 2. If it can't be evicted immediately, i.e., a dirty page pending
     *    writeback, add it to the second youngest generation.
     * 3. If it should be evicted first, e.g., cold and clean from
     *    folio_rotate_reclaimable(), add it to the oldest generation.
     * 4. Everything else falls between 2 & 3 above and is added to the
     *    second oldest generation if it's considered inactive, or the
     *    oldest generation otherwise. See lru_gen_is_active().
     */
    if (folio_test_active(folio))
        seq = max_seq;
    else if ((folio_test_reclaim(folio) &&
          (folio_test_dirty(folio) || folio_test_writeback(folio))))
        seq = max_seq - 1;
    else if (min_seq + MIN_NR_GENS >= max_seq)
        seq = min_seq;
    else
        seq = min_seq + 1;

    unsigned int gen = lru_gen_from_seq(seq);
    if (!gen_within_limits(gen)) {
        bpf_printk(
            "cache_ext: Invalid gen %d (min_seq=%lu, max_seq=%lu)\n",
            gen, min_seq, max_seq);
        return false;
    }
```

```
    // Update policy metadata
    struct folio_metadata metadata = { .accesses = 1, .gen = gen };
    __u64 key = (__u64)folio;
    int ret = bpf_map_update_elem(&folio_metadata_map, &key, &metadata,
                      BPF_ANY);
    if (ret != 0) {
        bpf_printk("cache_ext: Failed to save folio metadata\n");
        return false;
    }
    update_nr_pages_stat(lrugen, gen, folio_nr_pages(folio));

    // Update refaulted stats
    ret = folio_in_ghost(folio);
    if (ret >= 0) {
        int tier = ret;
        update_refaulted_stat(lrugen, tier, 1);
    }

    // lru_gen_update_size(lruvec, folio, -1, gen);
    ret = bpf_cache_ext_list_add(mglru_lists[gen], folio);
    if (ret != 0) {
        bpf_printk(
            "cache_ext: Failed to add folio to mglru_lists[%d]\n",
            gen);
        return false;
    }

    return true;
}

static inline bool should_run_aging(struct mglru_global_metadata *lrugen,
                    unsigned long max_seq)
{
    unsigned int gen;
    unsigned long old = 0;
    unsigned long young = 0;
    unsigned long total = 0;
    DEFINE_MIN_SEQ(lrugen);

    /* whether this lruvec is completely out of cold folios */
    if (min_seq + MIN_NR_GENS > max_seq) {
        return true;
    }

    unsigned long seq;
    int max_iter =  min(MAX_NR_GENS, max_seq - min_seq);
    for (int i = 0; i < max_iter; i++) {
        seq = min_seq + i;
        unsigned long size = 0;

        gen = lru_gen_from_seq(seq);

        size += max(read_nr_pages_stat(lrugen, gen), 0L);

        total += size;
        if (seq == max_seq)
            young += size;
        else if (seq + MIN_NR_GENS == max_seq)
            old += size;
    }

    /*
     * The aging tries to be lazy to reduce the overhead, while the eviction
     * stalls when the number of generations reaches MIN_NR_GENS. Hence, the
     * ideal number of generations is MIN_NR_GENS+1.
     */
    if (min_seq + MIN_NR_GENS < max_seq)
        return false;

    /*
     * It's also ideal to spread pages out evenly, i.e., 1/(MIN_NR_GENS+1)
     * of the total number of pages for each generation. A reasonable range
     * for this average portion is [1/MIN_NR_GENS, 1/(MIN_NR_GENS+2)]. The
     * aging cares about the upper bound of hot pages, while the eviction
     * cares about the lower bound of cold pages.
     */
    if (young * MIN_NR_GENS > total)
        return true;
    if (old * (MIN_NR_GENS + 2) < total)
        return true;

    return false;
}

static inline bool try_to_inc_min_seq(struct mglru_global_metadata *lrugen)
{
    // TODO: Check if the min_seq list is empty and if it is, increase min_seq.
    DEFINE_MIN_SEQ(lrugen);
    if (!gen_almost_empty(lrugen, min_seq)) {
        return false;
    }
    WRITE_ONCE(lrugen->min_seq, lrugen->min_seq + 1);
    reset_ctrl_pos(lrugen, true);
    return true;
}

static inline bool try_to_inc_max_seq(struct mglru_global_metadata *lrugen)
{
    // int prev, next;
    // spin_lock_irq(&lruvec->lru_lock);
    if (get_nr_gens(lrugen) == MAX_NR_GENS) {
        // Try to increase min_seq
        int ret = try_to_inc_min_seq(lrugen);
        if (!ret) {
            return false;
        }
    }
    // spin_unlock_irq(&lruvec->lru_lock);

    // TODO: Do we need this?
    /*
     * Update the active/inactive LRU sizes for compatibility. Both sides of
     * the current max_seq need to be covered, since max_seq+1 can overlap
     * with min_seq[LRU_GEN_ANON] if swapping is constrained. And if they do
     * overlap, cold/hot inversion happens.
     */
    // prev = lru_gen_from_seq(lrugen->max_seq - 1);
    // next = lru_gen_from_seq(lrugen->max_seq + 1);

    // for (type = 0; type < ANON_AND_FILE; type++) {
    //  for (zone = 0; zone < MAX_NR_ZONES; zone++) {
    //      enum lru_list lru = type * LRU_INACTIVE_FILE;
    //      long delta = lrugen->nr_pages[prev] -
    //               lrugen->nr_pages[next];

    //      if (!delta)
    //          continue;

    //      __update_lru_size(lruvec, lru, zone, delta);
    //      __update_lru_size(lruvec, lru + LRU_ACTIVE, zone, -delta);
    //  }
    // }

    reset_ctrl_pos(lrugen, false);

    // We don't use the timestamp metadata for our MGLRU
    /* make sure preceding modifications appear */
    WRITE_ONCE(lrugen->max_seq, lrugen->max_seq + 1);
    return true;

    // spin_unlock_irq(&lruvec->lru_lock);
}

////////////////////////////////////////////////////////////
//  ____    _    ____ _   _ _____     _______ ____   _   //
// / ___|  / \  / ___| | | | ____|   | ____\ \/ /_ _| |  //
// | |     / _ \| |   | |_| |  _|     |  _|  \  / | |    //
// | |___ / ___ \ |___|  _  | |___    | |___ /  \ | |    //
//  \____/_/   \_\____|_| |_|_____|___|_____/_/\_\___|    //
//                              |_____|                   //
////////////////////////////////////////////////////////////


/////////////
// Policy //
/////////////


struct eviction_metadata {
    __u64 curr_gen;
    __u64 next_gen;
    __u64 iter_reached;
    __u64 tier_threshold;
};
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, u32);
    __type(value, struct eviction_metadata);
} mglru_percpu_array SEC(".maps");
```

```
struct eviction_metadata * get_eviction_metadata() {
    u32 key = 0;
    return bpf_map_lookup_elem(&mglru_percpu_array, &key);
}

void set_eviction_metadata(struct eviction_metadata *eviction_meta) {
    u32 key = 0;
    int ret = bpf_map_update_elem(&mglru_percpu_array, &key, eviction_meta, BPF_ANY);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to update eviction metadata\n");
    }
}

inline bool is_folio_relevant(struct folio *folio)
{
    if (!folio) {
        return false;
    }
    if (folio->mapping == NULL) {
        return false;
    }
    if (folio->mapping->host == NULL) {
        return false;
    }
    bool res = inode_in_watchlist(folio->mapping->host->i_ino);
    return res;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_int;
    WRITE_ONCE(lrugen->max_seq, MIN_NR_GENS + 1);
    for (int i = 0; i < MAX_NR_GENS; i++) {
        __u64 list_ptr = bpf_cache_ext_ds_registry_new_list(memcg);
        if (list_ptr == 0) {
                    bpf_printk(
            "cache_ext: Failed to allocate list for gen %d\n",
            i);
            return -1;
        }
        mglru_lists[i] = list_ptr;
    }
    return 0;
}


// MGLRU iteration function. Logic is mainly ported from sort_folio.
static int mglru_iter_fn(int idx, struct cache_ext_list_node *a)
{
    // We skip the following cases from sort_folio since we asssume we don't
    // accept these folios in this policy:
    // - Unevictable folios
    // - Swap-backed folios
    // - Promoted folios (these only appear through PTE accesses,
    //                    fd-accessed folios are promoted based on their tier)

    struct mglru_global_metadata *lrugen;
    int key__ = 0;
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);
    if (!lrugen) {
        bpf_printk(
            "cache_ext: Failed to lookup lrugen metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }

    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (!eviction_meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get eviction metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }
    eviction_meta->iter_reached = idx;

    // Get folio metadata
    __u64 key = (__u64)a->folio;
    struct folio_metadata *meta =
        bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get metadata\n");
        // TODO: Maybe we should evict it instead?
        return CACHE_EXT_EVICT_NODE;
    }

    int tier_threshold = eviction_meta->tier_threshold;
    if (tier_threshold > MAX_NR_TIERS || tier_threshold < 0) {
        bpf_printk("cache_ext: Invalid tier threshold %d\n", tier_threshold);
    }

    // int tier_threshold = 2;
    int tier = lru_tier_from_refs(atomic_long_read(&meta->accesses));

    /* protected */
    if (tier > tier_threshold) {
        update_protected_stat(lrugen, tier, folio_nr_pages(a->folio));
        // promote to next gen
        // TODO: Update nr_pages stats
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }

    /* waiting for writeback */
    if (folio_test_locked(a->folio) || folio_test_writeback(a->folio) ||
        folio_test_dirty(a->folio)) {
        // promote to next gen
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_void;

    bool inc_max_seq_failed = false;
    bpf_spin_lock(&lrugen->lock);
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    if (should_run_aging(lrugen, max_seq)) {
        if (!try_to_inc_max_seq(lrugen)){
            // Workaround: Not allowed to call bpf_printk under spinlock
            inc_max_seq_failed = true;
        }
    }
    if (max_seq - min_seq > MIN_NR_GENS)
        try_to_inc_min_seq(lrugen);
    // Read min/max seq again
    min_seq = READ_ONCE(lrugen->min_seq);
    max_seq = READ_ONCE(lrugen->max_seq);
    int oldest_gen = lru_gen_from_seq(min_seq);
    volatile unsigned int next_gen = (oldest_gen + 1) % MAX_NR_GENS;
    bpf_spin_unlock(&lrugen->lock);

    if (inc_max_seq_failed) {
        bpf_printk("cache_ext: Failed to increment max_seq\n");
    }

    // captain-ahab: fixed threshold instead of the PID controller: protect only folios with >=16 refs (tier 3)
    int tier_threshold = MAX_NR_TIERS - 1;
    update_tier_selected_stat(lrugen, tier_threshold, 1);

    // Save eviction metadata for stats
    struct eviction_metadata ev_meta = {
        .curr_gen = oldest_gen,
        .next_gen = next_gen,
        .tier_threshold = tier_threshold,
    };
    set_eviction_metadata(&ev_meta);

    assert_valid_gen_0(next_gen);

    __u64 next_gen_list = mglru_lists[next_gen];
    __u64 oldest_gen_list = mglru_lists[oldest_gen];
    struct cache_ext_iterate_opts opts = {
        .continue_list = next_gen_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };


    int ret = bpf_cache_ext_list_iterate_extended(
        memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to iterate list\n");
        return;
    }
```

```
    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (eviction_meta == NULL) return;
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        min_seq = READ_ONCE(lrugen->min_seq);
        oldest_gen = lru_gen_from_seq(min_seq);
        next_gen = (oldest_gen + 1) % MAX_NR_GENS;
        __u64 next_gen_list = mglru_lists[next_gen];
        __u64 oldest_gen_list = mglru_lists[oldest_gen];
        struct cache_ext_iterate_opts opts = {
            .continue_list = next_gen_list,
            .continue_mode = CACHE_EXT_ITERATE_TAIL,
            .evict_list = CACHE_EXT_ITERATE_SELF,
            .evict_mode = CACHE_EXT_ITERATE_TAIL,
        };
        int ret = bpf_cache_ext_list_iterate_extended(
            memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
        if (ret < 0) {
            bpf_printk("cache_ext: Failed to iterate list\n");
            return;
        }
    }
    s64 success_evicted = eviction_ctx->nr_folios_to_evict;
    s64 failed_evicted = max(0, eviction_ctx->request_nr_folios_to_evict - eviction_ctx->nr_folios_to_evict);
    __sync_fetch_and_add(&lrugen->failed_evicted, failed_evicted);
    __sync_fetch_and_add(&lrugen->success_evicted, success_evicted);
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        bpf_printk("cache_ext: Failed to evict requested number of folios: %d/%d. Used list idx %d, list ptr: %p. Iterated %d folios",
                eviction_ctx->nr_folios_to_evict,
                eviction_ctx->request_nr_folios_to_evict,
                oldest_gen,
                oldest_gen_list,
                eviction_meta->iter_reached);
    }
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    lru_gen_add_folio(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    folio_inc_refs(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    DEFINE_LRUGEN_void;
    // Remove tracked metadata
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to delete folio metadata but not found in map.\n");
        return;
    }
    // Add ghost entry for refault detection
    int tier = lru_tier_from_refs(atomic_long_read(&metadata->accesses));
    insert_ghost_entry_for_folio(folio, tier);

    // Update generation page count
    update_evicted_stat(lrugen, tier, 1);
    update_nr_pages_stat(lrugen, metadata->gen, -folio_nr_pages(folio));

    bpf_map_delete_elem(&folio_metadata_map, &key);
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};

// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the MGLRU policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "evo_policy.skel.h"
#include "dir_watcher.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";

struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = { { "watch_dir", 'w', "DIR", 0,
                      "Directory to watch" },
                    { "cgroup_path", 'c', "PATH", 0,
                      "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
                    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 } };

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static volatile sig_atomic_t exiting;

static void sig_handler(int signo)
{
    exiting = 1;
}

int main(int argc, char **argv)
{
    int ret = 1;
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    int cgroup_fd = -1;
    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    {
        struct sigaction sa;
        memset(&sa, 0, sizeof(sa));
        sigemptyset(&sa.sa_mask);
        sa.sa_handler = sig_handler;
        if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
            perror("Failed to set up signal handling");
            return 1;
        }
```

```
}

// Parse command line arguments
struct cmdline_args args = { 0 };
struct argp argp = { options, parse_opt, 0, 0 };
argp_parse(&argp, argc, argv, 0, 0, &args);

// Validate arguments
if (args.watch_dir == NULL) {
    fprintf(stderr, "Missing required argument: watch_dir\n");
    return 1;
}

if (args.cgroup_path == NULL) {
    fprintf(stderr, "Missing required argument: cgroup_path\n");
    return 1;
}

// Does watch_dir exist?
if (access(args.watch_dir, F_OK) == -1) {
    fprintf(stderr, "Directory does not exist: %s\n",
        args.watch_dir);
    return 1;
}

// Get full path of watch_dir
char watch_dir_full_path[PATH_MAX];
if (realpath(args.watch_dir, watch_dir_full_path) == NULL) {
    perror("realpath");
    return 1;
}

// TODO: Enable longer length
if (strlen(watch_dir_full_path) > 128) {
    fprintf(stderr, "watch_dir path too long\n");
    return 1;
}

// Open cgroup directory early
cgroup_fd = open(args.cgroup_path, O_RDONLY);
if (cgroup_fd < 0) {
    perror("Failed to open cgroup path");
    return 1;
}

// Open skel
skel = evo_policy_bpf__open();
if (skel == NULL) {
    perror("Failed to open BPF skeleton");
    goto cleanup;
}

// Set watch_dir
skel->rodata->watch_dir_path_len = strlen(watch_dir_full_path);
strcpy(skel->rodata->watch_dir_path, watch_dir_full_path);

// Load programs
ret = evo_policy_bpf__load(skel);
if (ret) {
    perror("Failed to load BPF skeleton");
    goto cleanup;
}
// Initialize inode_watchlist map
ret = initialize_watch_dir_map(args.watch_dir,
                   bpf_map__fd(skel->maps.inode_watchlist), false);

// Attach cache_ext_ops to the specific cgroup
link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
if (link == NULL) {
    perror("Failed to attach cache_ext_ops to cgroup");
    goto cleanup;
}

// Attach probes
ret = evo_policy_bpf__attach(skel);
if (ret) {
    perror("Failed to attach BPF programs");
    goto cleanup;
}

printf("evo_policy (MGLRU) running. Press Ctrl+C to exit...\n");
fflush(stdout);
while (!exiting)
    sleep(1);
ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

pc318 (captain-nemo): best policy #53 35f7f9f5 — score 669.5, 669.5 ops/s

Agent's title: REPLICATION: reference mglru.c unchanged + 1 comment line (prev 657.4 on pc318 vs noop 615.7). Confirms or kills the +6.8% before tuning

Closest seed by line similarity: mglru 100%, sampling 34%, mru 33%. Lines that differ from mglru are highlighted.

```
// ==========================================================================
// SECTION: BPF KERNEL CODE
// ==========================================================================
// EVOLVE-BLOCK-START
// (replication run of reference mglru.c)
// MGLRU: a BPF port of Linux's multi-generational LRU. Folios sit in generations (by age)
// and tiers (by access frequency); aging advances the generation counters and eviction
// takes from the oldest generation, with a ghost map feeding refault information back.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ARRAY_SIZE(x) (sizeof(x) / sizeof((x)[0]))
#define INT64_MAX (9223372036854775807LL)

// #define DEBUG
#ifdef DEBUG
#define dbg_printk(fmt, ...) bpf_printk(fmt, ##__VA_ARGS__)
#else
#define dbg_printk(fmt, ...)
#endif

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define atomic_long_read(ptr) __sync_fetch_and_add(ptr, 0)
#define atomic_long_zero(ptr) __sync_fetch_and_and(ptr, 0)
#define atomic_long_store(ptr, val) __sync_lock_test_and_set(ptr, val)

#define DEFINE_MIN_SEQ(lrugen) \
    unsigned long min_seq = READ_ONCE(lrugen->min_seq)
#define DEFINE_MAX_SEQ(lrugen) \
    unsigned long max_seq = READ_ONCE(lrugen->max_seq)

/////////// 
// Maps //
///////////

#define MAX_NR_FOLIOS 4000000
#define MAX_NR_GHOST_ENTRIES 400000

struct folio_metadata {
    s64 accesses;
    s64 gen;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, __u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, MAX_NR_FOLIOS);
} folio_metadata_map SEC(".maps");

//////////////////
// Ghost Enties //
//////////////////

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    __uint(max_entries, MAX_NR_GHOST_ENTRIES);
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

static inline void insert_ghost_entry_for_folio(struct folio *folio, int tier) {
    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 ghost_val = (u8) tier;
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
            bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline int folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 *tier;
    tier = bpf_map_lookup_elem(&ghost_map, &key);
    if (tier == NULL) {
        return -1;
    }
    // TODO: handle non-ENOENT errors
    bpf_map_delete_elem(&ghost_map, &key);
    return (int)(*tier);
}

////////////////////////////////////////////////////////////////////////////////////
//  _  _____ ____ _____ _     _     ____ __  __ ____  _____ ____ ____  _____ ____  //
// | |/ / ____|  _ \| | \ | | ____| |     |  _ \ \/ /  | _ \ / _ \|  _ \_   _/ ___|  //
// | ' /|  _| | |_) |  \| |  _| | |      | || |\/| | |_) | | | | |_) || | \___ \  //
// | . \| |___|  _ <| |\  | |___| |___   | || |  | |  __/| |_| |  _ < | |  ___) | //
// |_|\_\_____|_| \_\_| \_|_____|_____| |___|_|  |_|_|    \___/|_| \_\|_| |____/  //
////////////////////////////////////////////////////////////////////////////////////

// Constants

#define MAX_NR_TIERS 4
#define MIN_NR_GENS 2
#define MAX_NR_GENS 4
#define NR_HIST_GENS 1
#define MIN_LRU_BATCH 64
// Global policy metadata
struct mglru_global_metadata {
    struct bpf_spin_lock lock;
    unsigned long max_seq;
    unsigned long min_seq;
    s64 evicted[MAX_NR_TIERS];
    s64 refaulted[MAX_NR_TIERS];
    s64 tier_selected[MAX_NR_TIERS];
    s64 success_evicted;
    s64 failed_evicted;
    unsigned long avg_refaulted[MAX_NR_TIERS];
    unsigned long avg_total[MAX_NR_TIERS];
    unsigned long protected[MAX_NR_TIERS - 1];
    long nr_pages[MAX_NR_GENS];
};

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, int);
    __type(value, struct mglru_global_metadata);
    __uint(max_entries, 1);
} mglru_global_metadata_map SEC(".maps");

#define DEFINE_LRUGEN_void                                                  \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                         \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return;                                                             \
    }

#define DEFINE_LRUGEN_int                                                   \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                         \
```

```
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return -1;                                                      \
    }

#define DEFINE_LRUGEN_bool                                                  \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                     \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return false;                                                   \
    }

// TODO: Hook these up with the eventual policy
// Helpers to update metadata

#define assert_valid_tier_0(tier_idx)                                   \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {               \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
                tier_idx);                                  \
        return;                                             \
    }

#define assert_valid_gen_0(gen_idx)                                         \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                        \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return;                                                     \
    }

#define assert_valid_tier_1(tier_idx)                                   \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {               \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
                tier_idx);                                  \
        return -1;                                          \
    }

#define assert_valid_gen_1(gen_idx)                                         \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                        \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return -1;                                                  \
    }

inline void update_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                  s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->refaulted[tier_idx], delta);
}

inline void update_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->evicted[tier_idx], delta);
}

inline void update_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx,
                 s64 delta)
{
    assert_valid_gen_0(gen_idx);
    __sync_fetch_and_add(&lrugen->nr_pages[gen_idx], delta);
}

inline void update_tier_selected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                 s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->tier_selected[tier_idx], delta);
}


inline s64 read_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->refaulted[tier_idx]));
}

inline s64 read_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->evicted[tier_idx]));
}

inline s64 read_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx)
{
    assert_valid_gen_1(gen_idx);
    return max(0, atomic_long_read(&lrugen->nr_pages[gen_idx]));
}

// Invoke when promoting a folio in the eviction iteration
// See: https://github.com/cache-ext/linux-cachestream/blob/c22ffcac6b53ef4054483070fb902895ef10fd12/mm/vmscan.c#L4941-L4952
inline void update_protected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                  s64 delta)
{
    __sync_fetch_and_add(&lrugen->protected[tier_idx - 1], delta);
}

// Gen lists
static __u64 mglru_lists[MAX_NR_GENS];

/******************************************************************************
 *                        PID controller
 ******************************************************************************/

/*
 * A feedback loop based on Proportional-Integral-Derivative (PID) controller.
 *
 * The P term is refaulted/(evicted+protected) from a tier in the generation
 * currently being evicted; the I term is the exponential moving average of the
 * P term over the generations previously evicted, using the smoothing factor
 * 1/2; the D term isn't supported.
 *
 * The setpoint (SP) is always the first tier of one type; the process variable
 * (PV) is either any tier of the other type or any other tier of the same
 * type.
 *
 * The error is the difference between the SP and the PV; the correction is to
 * turn off protection when SP>PV or turn on protection when SP<PV.
 *
 * For future optimizations:
 * 1. The D term may discount the other two terms over time so that long-lived
 *    generations can resist stale information.
 */

 // Avoid name conflict with kernel struct ctrl_pos if MGLRU is enabled
 struct ctrl_pos___x {
    unsigned long refaulted;
    unsigned long total;
    int gain;
};

static inline void read_ctrl_pos(struct mglru_global_metadata *lrugen, int tier,
                 int gain, struct ctrl_pos___x *pos)
{
    pos->refaulted = lrugen->avg_refaulted[tier] +
             __sync_fetch_and_add(&lrugen->refaulted[tier], 0);
    pos->total = lrugen->avg_total[tier] +
             __sync_fetch_and_add(&lrugen->evicted[tier], 0);
    if (tier)
        pos->total += lrugen->protected[tier - 1];
    pos->gain = gain;
}

// Needs the LRU lock. Why???
static inline void reset_ctrl_pos(struct mglru_global_metadata *lrugen, bool carryover)
{
    int tier;
    bool clear = carryover ? NR_HIST_GENS == 1 : NR_HIST_GENS > 1;

    // lockdep_assert_held(&lruvec->lru_lock);

    if (!carryover && !clear)
        return;

    for (tier = 0; tier < MAX_NR_TIERS; tier++) {
        if (carryover) {
            unsigned long sum;

            sum = lrugen->avg_refaulted[tier] +
                  atomic_long_read(&lrugen->refaulted[tier]);
            WRITE_ONCE(lrugen->avg_refaulted[tier], sum / 2);

            sum = lrugen->avg_total[tier] +
                  atomic_long_read(&lrugen->evicted[tier]);
            if (tier)
                sum += lrugen->protected[tier - 1];
            WRITE_ONCE(lrugen->avg_total[tier], sum / 2);
        }

        if (clear) {
            atomic_long_zero(&lrugen->refaulted[tier]);
```

```
            atomic_long_zero(&lrugen->evicted[tier]);
            if (tier)
                WRITE_ONCE(lrugen->protected[tier - 1], 0);
        }
    }
}

static inline bool positive_ctrl_err(struct ctrl_pos___x *sp, struct ctrl_pos___x *pv)
{
    /*
     * Return true if the PV has a limited number of refaults or a lower
     * refaulted/total than the SP.
     */
    return pv->refaulted < MIN_LRU_BATCH ||
           pv->refaulted * (sp->total + MIN_LRU_BATCH) * sp->gain <=
               (sp->refaulted + 1) * pv->total * pv->gain;
}

/////////////////////////////////////
// Utils that use the PID controller //
/////////////////////////////////////

static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
{
    int tier;
    struct ctrl_pos___x sp, pv;

    /*
     * To leave a margin for fluctuations, use a larger gain factor (1:2).
     * This value is chosen because any other tier would have at least twice
     * as many refaults as the firsfirst tier.
     */
    read_ctrl_pos(lrugen, 0, 1, &sp);
    for (tier = 1; tier < MAX_NR_TIERS; tier++) {
        read_ctrl_pos(lrugen, tier, 2, &pv);
        if (!positive_ctrl_err(&sp, &pv))
            break;
    }

    return tier - 1;
}

// static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
// {
//  // TODO: Swap this with the real implementation once we add refault tracking
//  // support.
//  return 0;
// }

static inline void folio_inc_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to inc refs but folio not found in map.\n");
        return;
    }
    __sync_fetch_and_add(&metadata->accesses, 1);
}

static inline int folio_lru_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata)
        return -1;

    return atomic_long_read(&metadata->accesses);
}

static inline unsigned int lru_gen_from_seq(unsigned long seq)
{
    return seq % MAX_NR_GENS;
}

static inline bool folio_test_active(struct folio *folio)
{
    return folio_lru_refs(folio) >= 2;
}

// Only counts up to 3.
```

```
static inline int order_base_2(int n)
{
    if (n <= 1) {
        return 0;
    } else if (n <= 3) {
        return 1;
    } else if (n <= 7) {
        return 2;
    } else if (n <= 8) {
        return 3;
    }
    return 3;
}

static inline int lru_tier_from_refs(int refs)
{
    /* see the comment in folio_lru_refs() */
    // kernel does some off-by-one calculation here, we remove the +1
    // return order_base_2(refs + 1);
    return order_base_2(refs);
}

static inline bool gen_within_limits(unsigned int gen)
{
    return 0 <= gen && gen <= MAX_NR_GENS;
}

static inline int get_nr_gens(struct mglru_global_metadata *lrugen)
{
    return lrugen->max_seq - lrugen->min_seq + 1;
}

static inline bool gen_almost_empty(struct mglru_global_metadata *lrugen,
                    int min_seq)
{
    int oldest_gen = lru_gen_from_seq(min_seq);
    int nr_folios = read_nr_pages_stat(lrugen, oldest_gen);
    int threshold = 4;
    return nr_folios <= threshold;
}

// TODO: This is supposed to run with a lock.
static inline bool lru_gen_add_folio(struct folio *folio)
{
    unsigned long seq;
    if (folio_test_unevictable(folio)) {
        bpf_printk("cache_ext: Unevictable folio\n");
        return false;
    }

    DEFINE_LRUGEN_bool;
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    /*
     * There are four common cases for this page:
     * 1. If it's hot, i.e., freshly faulted in, add it to the youngest
     *    generation, and it's protected over the rest below.
     * 2. If it can't be evicted immediately, i.e., a dirty page pending
     *    writeback, add it to the second youngest generation.
     * 3. If it should be evicted first, e.g., cold and clean from
     *    folio_rotate_reclaimable(), add it to the oldest generation.
     * 4. Everything else falls between 2 & 3 above and is added to the
     *    second oldest generation if it's considered inactive, or the
     *    oldest generation otherwise. See lru_gen_is_active().
     */
    if (folio_test_active(folio))
        seq = max_seq;
    else if ((folio_test_reclaim(folio) &&
          (folio_test_dirty(folio) || folio_test_writeback(folio))))
        seq = max_seq - 1;
    else if (min_seq + MIN_NR_GENS >= max_seq)
        seq = min_seq;
    else
        seq = min_seq + 1;

    unsigned int gen = lru_gen_from_seq(seq);
    if (!gen_within_limits(gen)) {
        bpf_printk(
            "cache_ext: Invalid gen %d (min_seq=%lu, max_seq=%lu)\n",
            gen, min_seq, max_seq);
        return false;
    }

    // Update policy metadata
    struct folio_metadata metadata = { .accesses = 1, .gen = gen };
    __u64 key = (__u64)folio;
```

```
    int ret = bpf_map_update_elem(&folio_metadata_map, &key, &metadata,
                      BPF_ANY);
    if (ret != 0) {
        bpf_printk("cache_ext: Failed to save folio metadata\n");
        return false;
    }
    update_nr_pages_stat(lrugen, gen, folio_nr_pages(folio));

    // Update refaulted stats
    ret = folio_in_ghost(folio);
    if (ret >= 0) {
        int tier = ret;
        update_refaulted_stat(lrugen, tier, 1);
    }

    // lru_gen_update_size(lruvec, folio, -1, gen);
    ret = bpf_cache_ext_list_add(mglru_lists[gen], folio);
    if (ret != 0) {
        bpf_printk(
            "cache_ext: Failed to add folio to mglru_lists[%d]\n",
            gen);
        return false;
    }

    return true;
}

static inline bool should_run_aging(struct mglru_global_metadata *lrugen,
                    unsigned long max_seq)
{
    unsigned int gen;
    unsigned long old = 0;
    unsigned long young = 0;
    unsigned long total = 0;
    DEFINE_MIN_SEQ(lrugen);

    /* whether this lruvec is completely out of cold folios */
    if (min_seq + MIN_NR_GENS > max_seq) {
        return true;
    }

    unsigned long seq;
    int max_iter =  min(MAX_NR_GENS, max_seq - min_seq);
    for (int i = 0; i < max_iter; i++) {
        seq = min_seq + i;
        unsigned long size = 0;

        gen = lru_gen_from_seq(seq);

        size += max(read_nr_pages_stat(lrugen, gen), 0L);

        total += size;
        if (seq == max_seq)
            young += size;
        else if (seq + MIN_NR_GENS == max_seq)
            old += size;
    }

    /*
     * The aging tries to be lazy to reduce the overhead, while the eviction
     * stalls when the number of generations reaches MIN_NR_GENS. Hence, the
     * ideal number of generations is MIN_NR_GENS+1.
     */
    if (min_seq + MIN_NR_GENS < max_seq)
        return false;

    /*
     * It's also ideal to spread pages out evenly, i.e., 1/(MIN_NR_GENS+1)
     * of the total number of pages for each generation. A reasonable range
     * for this average portion is [1/MIN_NR_GENS, 1/(MIN_NR_GENS+2)]. The
     * aging cares about the upper bound of hot pages, while the eviction
     * cares about the lower bound of cold pages.
     */
    if (young * MIN_NR_GENS > total)
        return true;
    if (old * (MIN_NR_GENS + 2) < total)
        return true;

    return false;
}

static inline bool try_to_inc_min_seq(struct mglru_global_metadata *lrugen)
{
    // TODO: Check if the min_seq list is empty and if it is, increase min_seq.
    DEFINE_MIN_SEQ(lrugen);
    if (!gen_almost_empty(lrugen, min_seq)) {
        return false;
    }
    WRITE_ONCE(lrugen->min_seq, lrugen->min_seq + 1);
    reset_ctrl_pos(lrugen, true);
    return true;
}

static inline bool try_to_inc_max_seq(struct mglru_global_metadata *lrugen)
{
    // int prev, next;
    // spin_lock_irq(&lruvec->lru_lock);
    if (get_nr_gens(lrugen) == MAX_NR_GENS) {
        // Try to increase min_seq
        int ret = try_to_inc_min_seq(lrugen);
        if (!ret) {
            return false;
        }
    }
    // spin_unlock_irq(&lruvec->lru_lock);

    // TODO: Do we need this?
    /*
     * Update the active/inactive LRU sizes for compatibility. Both sides of
     * the current max_seq need to be covered, since max_seq+1 can overlap
     * with min_seq[LRU_GEN_ANON] if swapping is constrained. And if they do
     * overlap, cold/hot inversion happens.
     */
    // prev = lru_gen_from_seq(lrugen->max_seq - 1);
    // next = lru_gen_from_seq(lrugen->max_seq + 1);

    // for (type = 0; type < ANON_AND_FILE; type++) {
    //  for (zone = 0; zone < MAX_NR_ZONES; zone++) {
    //      enum lru_list lru = type * LRU_INACTIVE_FILE;
    //      long delta = lrugen->nr_pages[prev] -
    //               lrugen->nr_pages[next];

    //      if (!delta)
    //          continue;

    //      __update_lru_size(lruvec, lru, zone, delta);
    //      __update_lru_size(lruvec, lru + LRU_ACTIVE, zone, -delta);
    //  }
    // }

    reset_ctrl_pos(lrugen, false);

    // We don't use the timestamp metadata for our MGLRU
    /* make sure preceding modifications appear */
    WRITE_ONCE(lrugen->max_seq, lrugen->max_seq + 1);
    return true;

    // spin_unlock_irq(&lruvec->lru_lock);
}

////////////////////////////////////////////////////////////
//  ____   _    ____ _   _ _____    ______ _______   __   //
// / ___| / \  / ___| | | | ____|   | ____\ \/ / _ \ / /_  //
// | |    / _ \| |   | |_| |  _|     |  _|  \  / | | | '_ \ //
// | |___ / ___ \ |___|  _  | |___    | |___ /  \ |_| | (_) | //
//  \____/_/   \_\____|_| |_|_____|___|_____/_/\_\___/ \___/  //
//                                |_____|                   //
////////////////////////////////////////////////////////////


////////////
// Policy //
////////////


struct eviction_metadata {
    __u64 curr_gen;
    __u64 next_gen;
    __u64 iter_reached;
    __u64 tier_threshold;
};
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, u32);
    __type(value, struct eviction_metadata);
} mglru_percpu_array SEC(".maps");


struct eviction_metadata * get_eviction_metadata() {
    u32 key = 0;
    return bpf_map_lookup_elem(&mglru_percpu_array, &key);
```

```
}

void set_eviction_metadata(struct eviction_metadata *eviction_meta) {
    u32 key = 0;
    int ret = bpf_map_update_elem(&mglru_percpu_array, &key, eviction_meta, BPF_ANY);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to update eviction metadata\n");
    }
}

inline bool is_folio_relevant(struct folio *folio)
{
    if (!folio) {
        return false;
    }
    if (folio->mapping == NULL) {
        return false;
    }
    if (folio->mapping->host == NULL) {
        return false;
    }
    bool res = inode_in_watchlist(folio->mapping->host->i_ino);
    return res;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_int;
    WRITE_ONCE(lrugen->max_seq, MIN_NR_GENS + 1);
    for (int i = 0; i < MAX_NR_GENS; i++) {
        __u64 list_ptr = bpf_cache_ext_ds_registry_new_list(memcg);
        if (list_ptr == 0) {
                    bpf_printk(
            "cache_ext: Failed to allocate list for gen %d\n",
            i);
            return -1;
        }
        mglru_lists[i] = list_ptr;
    }
    return 0;
}


// MGLRU iteration function. Logic is mainly ported from sort_folio.
static int mglru_iter_fn(int idx, struct cache_ext_list_node *a)
{
    // We skip the following cases from sort_folio since we asssume we don't
    // accept these folios in this policy:
    // - Unevictable folios
    // - Swap-backed folios
    // - Promoted folios (these only appear through PTE accesses,
    //                    fd-accessed folios are promoted based on their tier)

    struct mglru_global_metadata *lrugen;
    int key__ = 0;
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);
    if (!lrugen) {
        bpf_printk(
            "cache_ext: Failed to lookup lrugen metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }

    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (!eviction_meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get eviction metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }
    eviction_meta->iter_reached = idx;

    // Get folio metadata
    __u64 key = (__u64)a->folio;
    struct folio_metadata *meta =
        bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get metadata\n");
        // TODO: Maybe we should evict it instead?
        return CACHE_EXT_EVICT_NODE;
    }

    int tier_threshold = eviction_meta->tier_threshold;
    if (tier_threshold > MAX_NR_TIERS || tier_threshold < 0) {
        bpf_printk("cache_ext: Invalid tier threshold %d\n", tier_threshold);
    }
    // int tier_threshold = 2;
    int tier = lru_tier_from_refs(atomic_long_read(&meta->accesses));

    /* protected */
    if (tier > tier_threshold) {
        update_protected_stat(lrugen, tier, folio_nr_pages(a->folio));
        // promote to next gen
        // TODO: Update nr_pages stats
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }

    /* waiting for writeback */
    if (folio_test_locked(a->folio) || folio_test_writeback(a->folio) ||
        folio_test_dirty(a->folio)) {
        // promote to next gen
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_void;

    bool inc_max_seq_failed = false;
    bpf_spin_lock(&lrugen->lock);
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    if (should_run_aging(lrugen, max_seq)) {
        if (!try_to_inc_max_seq(lrugen)){
            // Workaround: Not allowed to call bpf_printk under spinlock
            inc_max_seq_failed = true;
        }
    }
    if (max_seq - min_seq > MIN_NR_GENS)
        try_to_inc_min_seq(lrugen);
    // Read min/max seq again
    min_seq = READ_ONCE(lrugen->min_seq);
    max_seq = READ_ONCE(lrugen->max_seq);
    int oldest_gen = lru_gen_from_seq(min_seq);
    volatile unsigned int next_gen = (oldest_gen + 1) % MAX_NR_GENS;
    bpf_spin_unlock(&lrugen->lock);

    if (inc_max_seq_failed) {
        bpf_printk("cache_ext: Failed to increment max_seq\n");
    }

    int tier_threshold = get_tier_idx(lrugen);
    update_tier_selected_stat(lrugen, tier_threshold, 1);

    // Save eviction metadata for stats
    struct eviction_metadata ev_meta = {
        .curr_gen = oldest_gen,
        .next_gen = next_gen,
        .tier_threshold = tier_threshold,
    };
    set_eviction_metadata(&ev_meta);

    assert_valid_gen_0(next_gen);

    __u64 next_gen_list = mglru_lists[next_gen];
    __u64 oldest_gen_list = mglru_lists[oldest_gen];
    struct cache_ext_iterate_opts opts = {
        .continue_list = next_gen_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };


    int ret = bpf_cache_ext_list_iterate_extended(
        memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to iterate list\n");
        return;
    }
    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (eviction_meta == NULL) return;
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        min_seq = READ_ONCE(lrugen->min_seq);
```

```
        oldest_gen = lru_gen_from_seq(min_seq);
        next_gen = (oldest_gen + 1) % MAX_NR_GENS;
        __u64 next_gen_list = mglru_lists[next_gen];
        __u64 oldest_gen_list = mglru_lists[oldest_gen];
        struct cache_ext_iterate_opts opts = {
            .continue_list = next_gen_list,
            .continue_mode = CACHE_EXT_ITERATE_TAIL,
            .evict_list = CACHE_EXT_ITERATE_SELF,
            .evict_mode = CACHE_EXT_ITERATE_TAIL,
        };
        int ret = bpf_cache_ext_list_iterate_extended(
            memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
        if (ret < 0) {
            bpf_printk("cache_ext: Failed to iterate list\n");
            return;
        }
    }
    s64 success_evicted = eviction_ctx->nr_folios_to_evict;
    s64 failed_evicted = max(0, eviction_ctx->request_nr_folios_to_evict - eviction_ctx->nr_folios_to_evict);
    __sync_fetch_and_add(&lrugen->failed_evicted, failed_evicted);
    __sync_fetch_and_add(&lrugen->success_evicted, success_evicted);
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        bpf_printk("cache_ext: Failed to evict requested number of folios: %d/%d. Used list idx %d, list ptr: %p. Iterated %d times",
                eviction_ctx->nr_folios_to_evict,
                eviction_ctx->request_nr_folios_to_evict,
                oldest_gen,
                oldest_gen_list,
                eviction_meta->iter_reached);
    }
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    lru_gen_add_folio(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    folio_inc_refs(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    DEFINE_LRUGEN_void;
    // Remove tracked metadata
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to delete folio metadata but not found in map.\n");
        return;
    }
    // Add ghost entry for refault detection
    int tier = lru_tier_from_refs(atomic_long_read(&metadata->accesses));
    insert_ghost_entry_for_folio(folio, tier);

    // Update generation page count
    update_evicted_stat(lrugen, tier, 1);
    update_nr_pages_stat(lrugen, metadata->gen, -folio_nr_pages(folio));

    bpf_map_delete_elem(&folio_metadata_map, &key);
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the MGLRU policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "evo_policy.skel.h"
#include "dir_watcher.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";

struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = { { "watch_dir", 'w', "DIR", 0,
                      "Directory to watch" },
                    { "cgroup_path", 'c', "PATH", 0,
                      "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
                    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 } };

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static volatile sig_atomic_t exiting;

static void sig_handler(int signo)
{
    exiting = 1;
}

int main(int argc, char **argv)
{
    int ret = 1;
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    int cgroup_fd = -1;
    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    {
        struct sigaction sa;
        memset(&sa, 0, sizeof(sa));
        sigemptyset(&sa.sa_mask);
        sa.sa_handler = sig_handler;
        if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
            perror("Failed to set up signal handling");
            return 1;
        }
    }

    // Parse command line arguments
    struct cmdline_args args = { 0 };
```

```
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, &args);

    // Validate arguments
    if (args.watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args.cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    // Does watch_dir exist?
    if (access(args.watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n",
            args.watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    char watch_dir_full_path[PATH_MAX];
    if (realpath(args.watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // TODO: Enable longer length
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    // Open skel
    skel = evo_policy_bpf__open();
    if (skel == NULL) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set watch_dir
    skel->rodata->watch_dir_path_len = strlen(watch_dir_full_path);
    strcpy(skel->rodata->watch_dir_path, watch_dir_full_path);

    // Load programs
    ret = evo_policy_bpf__load(skel);
    if (ret) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }

    // Initialize inode_watchlist map
    ret = initialize_watch_dir_map(args.watch_dir,
                      bpf_map__fd(skel->maps.inode_watchlist), false);

    // Attach cache_ext_ops to the specific cgroup
    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // Attach probes
    ret = evo_policy_bpf__attach(skel);
    if (ret) {
        perror("Failed to attach BPF programs");
        goto cleanup;
    }

    printf("evo_policy (MGLRU) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```